

Lab 01: Introduction to Software Testing

Software: Software is a set of instructions to perform some task. Software is used in many applications of the real world. Some of the examples are

- Application software, such as word processors
- Firmware in a embedded system
- Middleware, which controls and co-ordinates distributed systems
- System software such as operating systems
- Video Games
- Websites

All of these applications need to run without any error and provide a quality service to the user of the application. In this regard the software has to be tested for its accurate and correct working.

In order to make sure the released software is safe and functions as expected, the concept of **software quality** was introduced. It is often defined as “*the degree of conformance to explicit or implicit requirements and expectations*”. These so-called explicit and implicit expectations correspond to the two basic levels of software quality:

- **Functional** – the product’s compliance with functional (explicit) requirements and design specifications. This aspect focuses on the practical use of software, from the point of view of the user: its features, performance, ease of use, absence of defects.
- **Non-Functional** – system’s inner characteristics and architecture, i.e. structural (implicit) requirements. This includes the code maintainability, understandability, efficiency, and security.

Software quality measures how well software is designed (quality of design), and how well the software conforms to that design (quality of conformance). Software quality is a multidimensional quantity and is measurable. To do this, we need to divide and measure software quality in terms of quality attributes:

Software Quality Attributes:



ISO-9 126 (ISO, 2001) provides a hierarchical framework for quality definition, organized into quality characteristics and sub-characteristics

Functionality: *A set of attributes that bear on the existence of a set of functions and their specified properties. The functions are those that satisfy stated or implied needs. The sub-characteristics include:* - Suitability - Accuracy - Interoperability - Security

Reliability: *A set of attributes that bear on the capability of software to maintain its level of performance under stated conditions for a stated period of time. The sub-characteristics include:* - Maturity - Fault tolerance - Recoverability

Usability: *A set of attributes that bear on the effort needed for use, and on the individual assessment of such use, by a stated or implied set of users. The sub-characteristics include:* - Understandability - Learnability - Operability

Efficiency: *A set of attributes that bear on the relationship between the level of performance of the software and the amount of resources used, under stated conditions. The sub-characteristics include:* - Time behavior - Resource behavior

Maintainability: *A set of attributes that bear on the effort needed to make specified modifications. The sub-characteristics include:* - Analyzability - Changeability - Stability - Testability

Portability: *A set of attributes that bear on the ability of software to be transferred from one environment to another. The sub-characteristics include:* - Adaptability - Installability - Conformance – Replaceability

The structural quality of the software is usually hard to manage: It relies mostly on the expertise of the engineering team and can be assured through code review, analysis and refactoring. At the same time, functional aspect can be assured through a set of dedicated **quality management activities**, which includes quality assurance, quality control, and testing. Often used interchangeably, the three terms refer to slightly different aspects of software quality management. Despite a common goal of delivering a product of the best possible quality, both structurally and functionally, they use different approaches to this task.

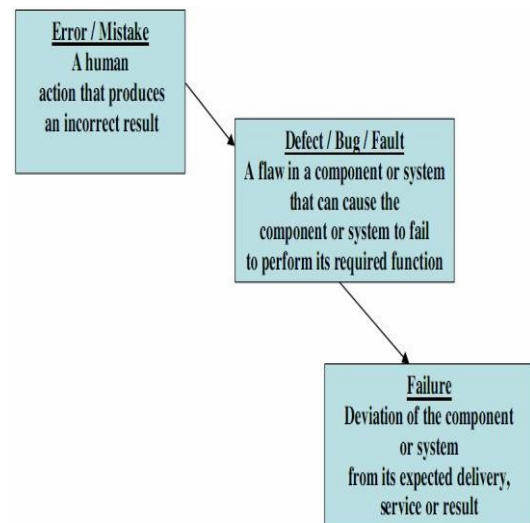


QA, QC and Testing in software development process

Functional Vs non-functional testing

Functional testing refers to tests that verify a specific action or function of the code. These are usually found in the code requirements documentation, although some development methodologies work from use cases or user stories. Functional tests tend to answer the question of "can the user do this" or "does this particular feature work".

Non-functional testing refers to aspects of the software that may not be related to a specific function or user action, such as scalability or security. Non-functional testing tends to answer such questions as "how many people can log in at once", or "how easy is it to hack this software".



Error, Fault and Failure:

Humans make errors in their thoughts, actions, and in the products that might result from their actions. Errors occur in the process of writing a program. A programmer makes an error (mistake), which results in a defect (fault, bug) in the software source code. If this defect is executed, in certain situations the system will produce wrong results, causing a failure. Not all defects will necessarily result in failures. For example, defects in dead code will never result in failures. A defect can turn into a failure when the environment is changed. Examples of these changes in environment include the software being run on a new hardware platform, alterations in source data or interacting with different software. A single defect may result in a wide range of failure symptoms.

Not all software defects are caused by coding errors. One common source of expensive defect is caused by requirement gaps, e.g., unrecognized requirements that result in errors of omission by the program designer. A common source of requirement's gaps is non-functional requirements such as testability, scalability, maintainability, usability, performance, and security.

Errors – Examples

- Incorrect usage of software by users
- Bad architecture and design by architects and designers
- Bad programming by developers
- Inadequate testing by testers
- Wrong build using incorrect configuration items by Build Team Member

Fault - Examples

- A fault is the manifestation of one or more errors

- An incorrect statement
- Wrong data type
- Wrong mathematical formula in design document
- Missing functionality in the system

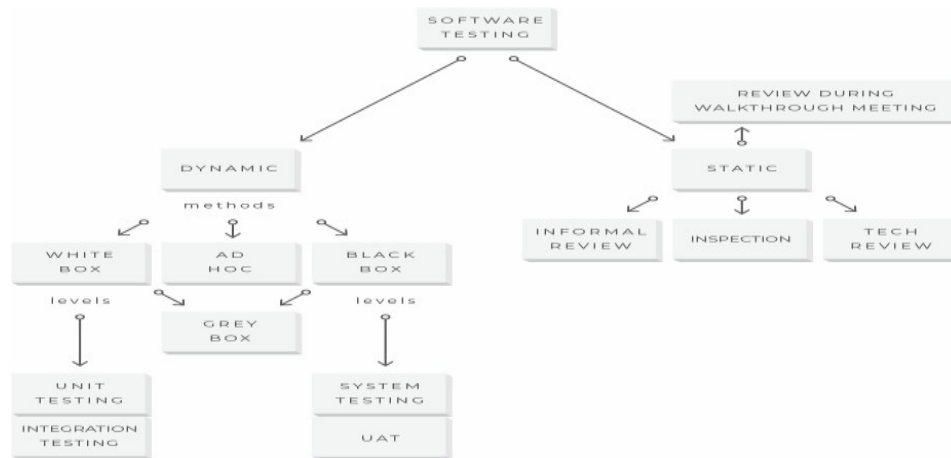
Failure

A failure occurs when a faulty piece of code is executed leading to incorrect state that propagates to the program's output. The following figure tells us how Error made by human will result in failure of the software.

Software Testing Objectives: Testing is done to fulfill certain objectives:

- To describe the principles of system and component testing
- To describe strategies for generating system test cases
- To understand the essential characteristics of tool used for test automation
- To find or prevent defects
- To determine that software products satisfy specified requirements
- Ensuring that a system is ready for use
- Gaining confidence that it works
- Providing information about the level of quality
- Determining user acceptability

Software Testing Types:



❖ **Static testing** initially examines the source code and software project documents to catch and prevent defects early in the software testing life cycle. Also called non-execution technique or verification testing, static testing could be performed as inspections, informal and technical reviews, or reviews during walkthrough meetings. Informal review is a cheap testing variant that a QA analyst can conduct anytime during the project. Inspection, also called a formal review, is planned and controlled by the moderator. During the review meeting, errors found by QA analysts are discussed and documented in the review report.

❖ **Dynamic testing** : software is tested during execution. This whitepaper has the most focus on

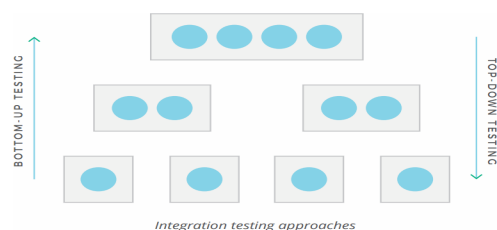
the dynamic testing process as a practical and most commonly used way to validate code behavior. Dynamic testing can be described by methods, levels, and types of underlying QA activities. Let's have a closer look at this segment of the dynamic testing process.

Software testing levels describe stages of software development when testing is conducted. That said, there are four progressive testing levels based on the area they focus on the software development process: unit testing, integration testing, system testing, and user acceptance testing (UAT).



- **Component/Unit Testing:** The smallest testable part of the software system is often referred to as a unit. Therefore, this testing level is aimed at examining every single unit of a software system in order to make sure that it meets the original requirements and functions as expected. Unit testing is commonly performed early in the development process by the engineers themselves, not the testing team.
- **Integration Testing:** The objective of the next testing level is to verify whether the combined units work well together as a group. Integration testing is aimed at detecting the flaws in the interactions between the units within a module. There are two main approaches to this testing: bottom-up and top-down methods. The bottom-up integration testing starts with unit tests, successively increasing the complexity of the software modules under test. The top-down method takes the opposite approach, focusing on high-level combinations first and examining the simple ones later.
- **System Testing:** At this level, a complete software system is tested as a whole. This stage serves to verify the product's compliance with the functional and technical requirements and overall quality standards. System testing should be performed by a highly professional testing team in an environment as close to the real business use scenario as possible.
- **User Acceptance Testing:** This is the last stage of the testing process, where the product is validated against the end user requirements and for accuracy. This final step helps the team decide if the product is ready to be shipped or not. While small issues should be detected and resolved earlier in the process, this testing level focuses on overall system quality, from content and UI to performance issues. The acceptance stage might be followed by an alpha and beta testing, allowing a small number of actual users to try out the software before it is officially released.

Software testing methods are the ways the tests are conducted. They include black box testing, white box testing, and gray box testing, and ad hoc testing.



- **Black Box Testing:** This method gets its name because a QA engineer focuses on the inputs and the expected outputs without knowing how the application works internally and how these inputs are processed. The purpose of this method is to check the functionality of the software making sure that it works correctly and

meets user demands. This method can be applied to any testing level but is used mostly for system and user acceptance testing. A QA specialist doesn't consider the internal processes of the product while conducting a test

- **White Box Testing:** Unlike black box testing, this method requires profound knowledge of the code as it entails testing of some structural part of the application. Therefore, generally, the developers directly involved in writing code are responsible for this type of testing. The purpose of white box testing is to enhance security, the flow of inputs/outputs through the application, and to improve design and usability. This method is mainly used at the unit and integration testing levels.
- **Gray Box Testing:** This method is a combination of the previous two, since it involves testing of both functional and structural parts of the application. Using this method, an experienced tester has partial knowledge of the internal application structure and based on this knowledge can design test cases while still testing from the black-box perspective. This method is mostly applicable to the integration testing level.

Software Testing Life Cycle: Software testing life cycle identifies what test activities to carry out and when (what is the best time) to accomplish those test activities. Even though testing differs between organizations, there is a testing life cycle. Software Testing Life Cycle consists of six (generic) phases:

- Test Planning,
- Test Analysis,
- Test Design,
- Construction and verification,
- Testing Cycles,
- Final Testing and Implementation and
- Post Implementation.

Test Planning : This is the phase where the Project Manager has to decide what things need to be tested, do I have the appropriate budget etc. Naturally proper planning at this stage would greatly reduce the risk of low quality software. This planning will be an ongoing process with no end point.

Activities at this stage would include preparation of a high level test plan-(according to IEEE test plan template The Software Test Plan (STP) is designed to prescribe the scope, approach, resources, and schedule of all testing activities. The plan must identify the items to be tested, the features to be tested, the types of testing to be performed, the personnel responsible for testing, the resources and schedule required to complete testing, and the risks associated with the plan.). Almost all of the activities done during this stage are included in this software test plan and revolve around a test plan.

Test Analysis : Once test plan is made and decided upon, next step is to delve little more into the project and decide what types of testing should be carried out at different stages of SDLC, do we need or plan to automate, if yes then when the appropriate time to automate is, what type of specific documentation we need for testing. Proper and regular meetings should be held between testing

teams, project managers, and development teams, Business Analysts to check the progress of things which will give a fair idea of the movement of the project and ensure the completeness of the test plan created in the planning phase, which will further help in enhancing the right testing strategy created earlier. We will start creating test case formats and test cases itself. In this stage we need to develop Functional validation matrix based on Business Requirements to ensure that all system requirements are covered by one or more test cases, identify which test cases to automate, begin review of documentation, i.e. Functional Design, Business Requirements, Product Specifications, Product Externals etc. We also have to define areas for Stress and Performance testing.

Test Design :Test plans and cases which were developed in the analysis phase are revised. Functional validation matrix is also revised and finalized. In this stage risk assessment criteria is developed. If you have thought of automation then you have to select which test cases to automate and begin writing scripts for them. Test data is prepared. Standards for unit testing and pass / fail criteria are defined here. Schedule for testing is revised (if necessary) & finalized and test environment is prepared.

Construction and verification :In this phase we have to complete all the test plans, test cases, complete the scripting of the automated test cases, Stress and Performance testing plans needs to be completed. We have to support the development team in their unit testing phase. And obviously bug reporting would be done as when the bugs are found. Integration tests are performed and errors (if any) are reported.

Testing Cycles :In this phase we have to complete testing cycles until test cases are executed without errors or a predefined condition is reached. Run test cases --> Report Bugs --> revise test cases (if needed) --> add new test cases (if needed) --> bug fixing --> retesting (test cycle 2, test cycle 3....).

Final Testing and Implementation :In this we have to execute remaining stress and performance test cases, documentation for testing is completed / updated, provide and complete different matrices for testing. Acceptance, load and recovery testing will also be conducted and the application needs to be verified under production conditions.

Post Implementation :In this phase, the testing process is evaluated and lessons learnt from that testing process are documented. Line of attack to prevent similar problems in future projects is identified. Create plans to improve the processes. The recording of new errors and enhancements is an ongoing process. Cleaning up of the test environment is done and test machines are restored to base lines in this stage.

EXERCISE:

1. Name the stages involved in the software testing life cycle, differentiate test planning and test analysis.

2. Differentiate between white box and black box testing techniques. Which technique do you prefer where you are asked to perform testing at component level and why? Explain with suitable examples.
3. Explain software testing levels.
4. Write down some advantages of software testing.
5. Write a test set for copying a file XYZ from folder A to folder B. (at least 5 test cases)

Lab 02: Manual Testing: Test Cases Creation

/*Write Test Cases for Facebook login and signup pages

Test case/data

- A set of test inputs, execution conditions, and expected results developed for a particular objective, such as to exercise a particular program path or to verify compliance with a specific requirement.
- A test case is a pair consisting of test data to be input to the program and the expected output. The test data is a set of values, one for each input variable.
- A test set is a collection of zero or more test cases.
- A test case in software engineering is a set of conditions or variables under which a tester will determine whether an application or software system is working correctly or not. The mechanism for determining whether a software program or system has passed or failed such a test is known as a test oracle. In some settings, an oracle could be a requirement or use case, while in others it could be a heuristic. It may take many test cases to determine that a software program or system is functioning correctly. Test cases are often referred to as test scripts, particularly when written. Written test cases are usually collected into test suites.

Typical written test case format

A test case is usually a single step, or occasionally a sequence of steps, to test the correct behavior/ functionalities, features of an application. An expected result or expected outcome is usually given. Additional information that may be included:

- test case ID
- test case description
- test step or order of execution number
- related requirement(s)
- depth
- test category
- author
- Additional fields that may be included and completed when the tests are executed:
 - pass/fail
 - remarks/comments

These steps can be stored in a word processor document, spreadsheet, database or other common repository. In a database system, you may also be able to see past test results and who generated the results and the system configuration used to generate those results. These past results would usually be stored in a separate table. Test suites may also contain

- Test summary
- Configuration

Template:

Project Name:

Test Case Template example

Test Case ID: google_10

Test Designed by: <Name>

Test Priority (Low/Medium/High): Med

Test Designed date: <Date>

Module Name: Google login screen

Test Executed by: <Name>

Test Title: Verify login module

Test Execution date: <Date>

Description: Test the Google login page

Pre-conditions: User has valid username and password

Dependencies:

S.no	Test Case	Test Steps	Test Data	Expected Output	Actual Output	Pass/Fail	Comments?
Tc_001	Verify URL	1. Open Browser 2. Type URL 3. Press Enter	https://www.gmail.com	Page should be loaded	loaded	pass	

Post Condition: User is validated with database and successfully login to account. The account session details are logged in database.

- ❖ **Test case ID:** Unique ID for each test case. Follow some convention to indicate types of test. E.g. 'TC_UI_1' indicating 'user interface test case #1'.

- ❖ **Test priority (Low/Medium/High):** This is useful while test execution. Test priority for business rules and functional test cases can be medium or higher whereas minor user interface cases can be low priority. Test priority should be set by reviewer.
 - ❖ **Module Name** – Mention name of main module or sub module.
 - ❖ **Test Designed By:** Name of tester
 - ❖ **Test Designed Date:** Date when wrote
 - ❖ **Test Executed By:** Name of tester who executed this test. To be filled after test execution.
 - ❖ **Test Execution Date:** Date when test executed.
 - ❖ **Test Title/Name:** Test case title. E.g. verify login page with valid username and password.
 - ❖ **Test Summary/Description:** Describe test objective in brief.
 - ❖ **Pre-condition:** Any prerequisite that must be fulfilled before execution of this test case. List all pre-conditions in order to successfully execute this test case.
 - ❖ **Dependencies:** Mention any dependencies on other test cases or test requirement.
 - ❖ **Test Steps:** List all test execution steps in detail. Write test steps in the order in which these should be executed. Make sure to provide as much details as you can. Tip – to efficiently manage test case with lesser number of fields use this field to describe test conditions, test data and user roles for running test.
 - ❖ **Test Data:** Use of test data as an input for this test case. You can provide different data sets with exact values to be used as an input.
 - ❖ **Expected Result:** What should be the system output after test execution? Describe the expected result in detail including message/error that should be displayed on screen.
 - ❖ **Post-condition:** What should be the state of the system after executing this test case?
 - ❖ **Actual result:** Actual test result should be filled after test execution. Describe system behavior after test execution.
 - ❖ **Status (Pass/Fail):** If actual result is not as per the expected result mark this test as failed. Otherwise update as passed.
 - ❖ **Notes/Comments/Questions:** To support above fields if there are some special conditions which can't be described in any of the above fields or there are questions related to expected or actual results mention those here.
- Add following fields if necessary:***
- ❖ **Defect ID/Link:** If test status is fail, then include the link to defect log or mention the defect number.
 - ❖ **Test Type/Keywords:** This field can be used to classify tests based on test types. E.g. functional, usability, business rules etc.
 - ❖ **Requirements:** Requirements for which this test case is being written. Preferably the exact section number of the requirement doc.
 - ❖ **Attachments/References:** This field is useful for complex test scenarios. To explain test steps or expected result using a visio diagram as a reference. Provide the link or location to the actual path of the diagram or document.
 - ❖ **Automation? (Yes/No):** Whether this test case is automated or not. Useful to track automation status when test cases are automated.

Example:

Project Name:	Google Email								
Module Name:	Login								
Reference Document:	If any								
Created by:	Your Name								
Date of creation:	DD-MMM-YY								
Date of review:	DD-MMM-YY								
TEST CASE ID	TEST SCENARIO	TEST CASE	PRE-CONDITION	TEST STEPS	TEST DATA	EXPECTED RESULT	POST CONDITION	ACTUAL RESULT	STATUS (PASS/FAIL)
TC_LOGIN_001	Verify the login of Gmail	Enter valid User Name and valid Password	1. Need a valid Gmail Account to do login	1. Enter User Name	<Valid User Name>	Successful login	Gmail inbox is shown		
				2. Enter Password	<Valid Password>				
				3. Click "Login" button					
TC_LOGIN_001	Verify the login of Gmail	Enter valid User Name and invalid Password	1. Need a valid Gmail Account to do login	1. Enter User Name	<Valid User Name>	A message "The email and password you entered don't match" is shown			
				2. Enter Password	<Invalid Password>				
				3. Click "Login" button					
TC_LOGIN_001	Verify the login of Gmail	Enter invalid User Name and valid Password	1. Need a valid Gmail Account to do login	1. Enter User Name	<Invalid User Name>	A message "The email and password you entered don't match" is shown			
				2. Enter Password	<Valid Password>				
				3. Click "Login" button					
TC_LOGIN_001	Verify the login of Gmail	Enter invalid User Name and invalid Password	1. Need a valid Gmail Account to do login	1. Enter User Name	<Invalid User Name>	A message "The email and password you entered don't match" is shown			
				2. Enter Password	<Invalid Password>				
				3. Click "Login" button					

EXERCISE: write detailed Test cases for face book Login and signup pages by using template
/*Attach printout here */

Lab 03: white box Testing

White-box testing is also known as structure testing or glass-box testing. It focuses on the procedural details (or internal structure) of the software when generating test data and test cases. The derived test cases guarantee that all independent paths within a module have been exercised at least once and that all logical decisions have been exercised on their true and false sides. White-box testing is performed early in the testing process. White-box testing is mostly used at unit level testing.

What do you verify in White Box Testing?

White box testing involves the testing of the software code for the following:

- Internal security holes
- Broken or poorly structured paths in the coding processes
- The flow of specific inputs through the code
- Expected output
- The functionality of conditional loops
- Testing of each statement, object, and function on an individual basis

The testing can be done at system, integration and unit levels of software development. One of the basic goals of white box testing is to verify a working flow for an application. It involves testing a series of predefined inputs against expected or desired outputs so that when a specific input does not result in the expected output, you have encountered a bug.

Working process of white box testing:

- **Input:** Requirements, Functional specifications, design documents, source code.
- **Processing:** Performing risk analysis for guiding through the entire process.
- **Proper test planning:** Designing test cases so as to cover entire code. Execute rinse-repeat until error-free software is reached. Also, the results are communicated.
- **Output:** Preparing final report of the entire testing process.

How do you perform White Box Testing?

To give you a simplified explanation of white box testing, we have divided it into **two basic steps**. This is what testers do when testing an application using the white box testing technique:

Step 1. Understand the Source Code

The first thing a tester will often do is learn and understand the source code of the application. Since white box testing involves the testing of the inner workings of an application, the tester must be very knowledgeable in the programming languages used in the applications they are testing. Also, the testing person must be highly aware of secure coding practices. Security is often one of the primary objectives of testing software. The tester should be able to find security issues and prevent attacks from hackers and naive users who might inject malicious code into the application either knowingly or unknowingly.

Step 2. Create Test Cases and Execute

The second basic step to white box testing involves testing the application's source code for proper flow and structure. One way is by writing more code to test the application's source code. The tester will develop little tests for each process or series of processes in the application. This method requires that the tester must have intimate knowledge of the code and is often done by the

developer. Other methods include Manual Testing, trial, and error testing and the use of testing tools

Example:

Consider the following piece of code

```
Printme (int a, int b) {                                Printme is a function
    int result = a+ b;
    If (result> 0)
        Print ("Positive", result)
    Else
        Print ("Negative", result)
}                                                        End of the source code
```

The goal of WhiteBox testing in software engineering is to verify all the decision branches, loops, statements in the code. To exercise the statements in the above white box testing example, WhiteBox test cases would be

- A = 1, B = 1
- A = -1, B = -3

White Box Testing Techniques

A major White box testing technique is Code Coverage analysis. Code Coverage analysis eliminates gaps in a Test Case suite. It identifies areas of a program that are not exercised by a set of test cases. Once gaps are identified, you create test cases to verify untested parts of the code, thereby increasing the quality of the software product. Below are a few coverage analysis techniques a box tester can use:

Statement Coverage: - This technique requires every possible statement in the code to be tested at least once during the testing process of software engineering. It ensures every single line of code is tested.

Branch Coverage: - This technique checks every possible path (if-else and other conditional loops) of a software application. It ensures every branch (e.g. true or false) is tested.

Path Coverage: – Path testing is a structural testing method that involves using the source code of a program in order to find every possible executable path. It helps to determine all faults lying within a piece of code. This method is designed to execute all or selected path through a computer program. It ensures all possible paths are tested.

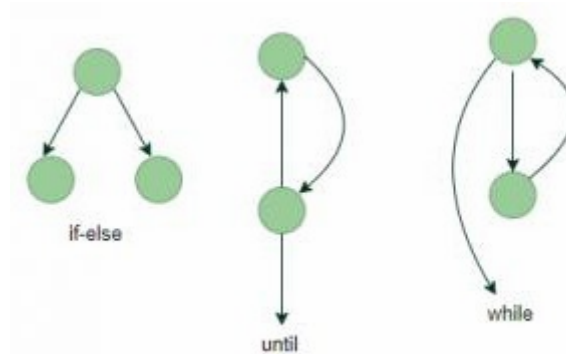
Basis/Independent Path Testing: In this technique, control flow graphs are made from code or flowchart and then Cyclomatic complexity is calculated which defines the number of independent paths so that the minimal number of test cases can be designed for each independent path.

Steps:

1. Make the corresponding control flow graph
2. Calculate the cyclomatic complexity
3. Find the independent paths
4. Design test cases corresponding to each independent path

Flow graph notation: It is a directed graph consisting of nodes and edges. Each node represents a sequence of statements, or a decision point. A predicate node is the one that represents a decision

point that contains a condition after which the graph splits. Regions are bounded by nodes and edges.



Cyclomatic Complexity: It is a measure of the logical complexity of the software and is used to define the number of independent paths. For a graph G , $V(G)$ is its cyclomatic complexity.

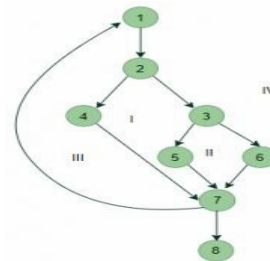
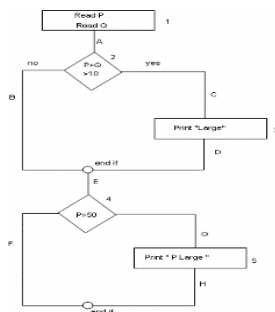
Calculating $V(G)$:

$V(G) = P + 1$, where P is the number of predicate nodes in the flow graph

$V(G) = E - N + 2$, where E is the number of edges and N is the total number of nodes

$V(G)$ = Number of non-overlapping regions in the graph

Example:



$V(G) = 4$ (Using any of the above formulae)

No of independent paths = 4

#P1: 1 – 2 – 4 – 7 – 8

#P2: 1 – 2 – 3 – 5 – 7 – 8

#P3: 1 – 2 – 3 – 6 – 7 – 8

#P4: 1 – 2 – 4 – 7 – 1 – ... – 7 – 8

Example :

- Read P
- Read Q
- IF P+Q > 100 THEN
- Print “Large”
- ENDIF
- If P > 50 THEN
- Print “P Large”
- ENDIF

Number of test cases required:

- ❖ **Statement Coverage:** 1A-2C-3D-E-4G-5H = 1
- ❖ **Branch Coverage:** 1A-2C-3D-E-4G-5H , 1A-2B-E-4F = 2
- ❖ **Path Coverage:** 1A-2B-E-4F, 1A-2B-E-4G-5H, 1A-2C-3D-E-4G-5H, 1A-2C-3D-E-4F=4

Code Coverage Calculation:

Statements Coverage: Simply, these are what you have in the boxes and diamond shapes.

You can cover all the statements in the flowchart by writing **1 Test Case** that follows the following route 1A-2C-3D-E-4G-5H. (p=60,q=50)

Let us say your test case covers the following route - 1A -> 2B -> E -> 4F. There are **total of 5 Statements** in your flowchart, You **cover only 3 Statements**. Now the Statement Coverage will be - **(3/5)*100**. Your Test Case, therefore, has **60% of Statement Coverage**

Branches/Decisions Coverage: Decisions that you can take in the process flow diagram (For example, if you consider the statement 2, there are two branches to it) You can cover all the branches in the flowchart by writing **2 Test Cases** that follow the following two routes 1A-2C-3D-E-4G-5H, 1A-2B-E-4F. (p=60,Q=50 , p=30,q=30)

Let us say your test case covers the following route - 1A -> 2B -> E -> 4F. There are **total of 4 Decision** in your flowchart (2 decisions for Statement no 2 and 2 decisions for Statement no 4). You **cover only 2 Branches/Decisions**. Now the Branch/Decision Coverage will be - **(2/4)*100**. Your Test Case, therefore, has **50% of Branch Coverage**

Paths Coverage: paths that you can take to travel in the flow chart from the start to the end. You can cover all the paths in the flowchart by writing **4 Test Cases** that follow the following four routes 1A-2B-E-4F, 1A-2B-E-4G-5H, 1A-2C-3D-E-4G-5H, 1A-2C-3D-E-4F. Therefore, the Path Coverage is 4. Let us say your test case covers the following route - 1A -> 2B -> E -> 4F. There are **total of 4 Paths** in your flowchart. You **cover only 1 Path**. Now the Path Coverage will be - **(1/4)*100**. Your Test Case, therefore, has **25% of Path Coverage**.

Apart from above, there are numerous coverage types such as Condition Coverage, Multiple Condition Coverage, Function Coverage etc. Each technique has its own merits and attempts to test (cover) all parts of software code.

Advantages of White Box Testing

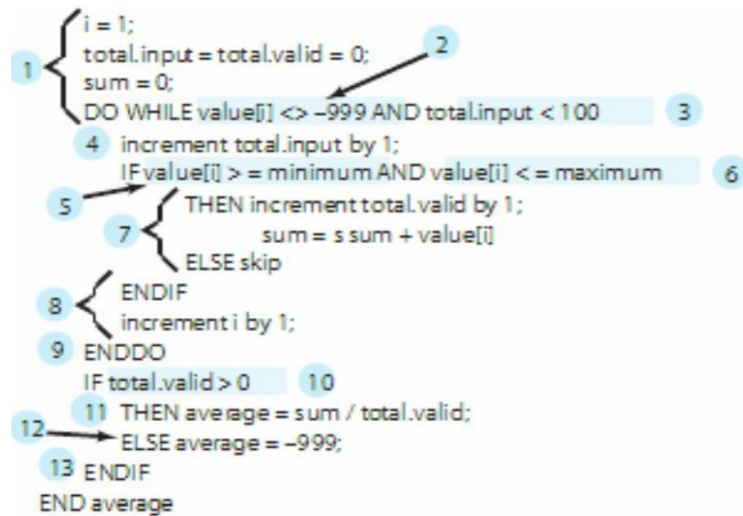
- Code optimization by finding hidden errors.
- White box tests cases can be easily automated.
- Testing is more thorough as all code paths are usually covered.
- Testing can start early in SDLC even if GUI is not available.

Disadvantages of White Box Testing

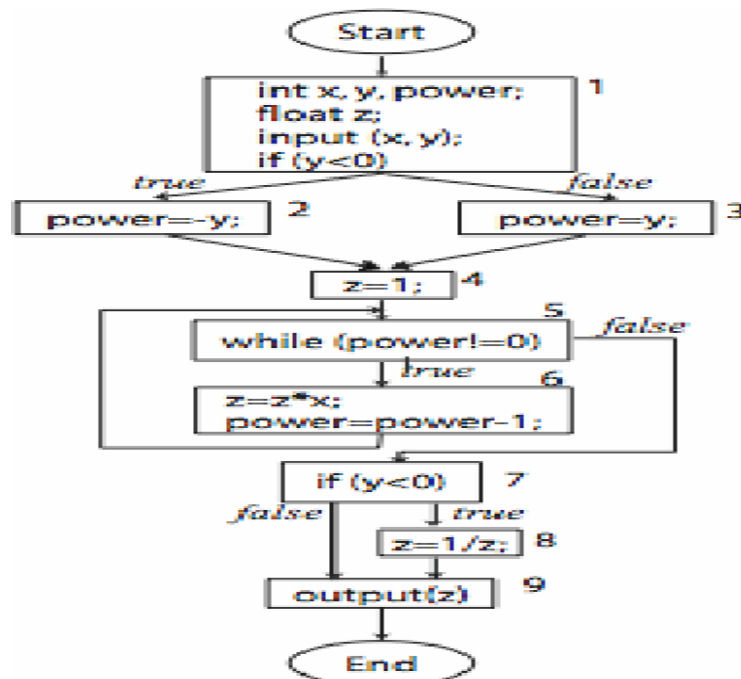
- White box testing can be quite complex and expensive.
- Developers who usually execute white box test cases detest it. The white box testing by developers is not detailed can lead to production errors.
- White box testing requires professional resources, with a detailed understanding of programming and implementation.
- White-box testing is time-consuming, bigger programming applications take the time to test fully.

EXERCISE:

1. Consider the code given below. Calculate cyclomatic complexity and no. of linearly independent paths and Design Testcases for each path. Attach all printout here



2. Consider the flow chart given below and design test cases by using statement coverage method, AND Independent path method also execute test cases using template (Attach All printout here).



3. Write a program (Any Language) to find odd or even number. Design and execute test cases using branch coverage method using template /*attach printout here*/

4. Write a program (Any Language) to perform addition, subtraction, multiplication and division. Observe the output and Test it using any one of white box technique .Attached all printout here.

- Description:**

Project Name:	Basic-Calculator(say)
Test Suite ID:	Calculate_TESEX(Your roll no. here)
Test Title:	Test the basic operators in Calculator
Test Priority:	Medium
Module Name:	Do_Calculation
Designed By:	/*Name
Designed Date:	/*date
Executed By:	/*name
Executed Date:	/*date
Description of Test:	Verify the addition, subtraction, multiplication and division of two numbers(Also test division by zero)
PREREQUISITES:	
Pre-Condition:	Not required
Dependencies:	No dependencies in this test.

- Test Cases:**

S.no	Test Case	Test Steps	Test Data	Expected Output	Actual Output	Pass/ Fail

Lab 04a: Equivalence Class Partitioning(ECP)

Equivalence partitioning is a software testing technique that divides the input data of a software unit into partition of data from which test cases can be derived. In principle, test cases are designed to cover each partition at least once. This technique tries to define test case that uncovers classes of errors, thereby reducing the total number of test cases that must be developed.

Test cases are designed for equivalence data class. The equivalence partitions are frequently derived from the requirements specification for input data that influence the processing of the test object. A use of this method reduces the time necessary for testing software using less and effective test cases. It can be used at any level of software for testing and is preferably a good technique to use first. In this technique, only one condition to be tested from each partition. Because we assume that, all the conditions in one partition behave in the same manner by the software. In a partition, if one condition works other will definitely work. Likewise we assume that, if one of the condition does not work then none of the conditions in that partition will work.

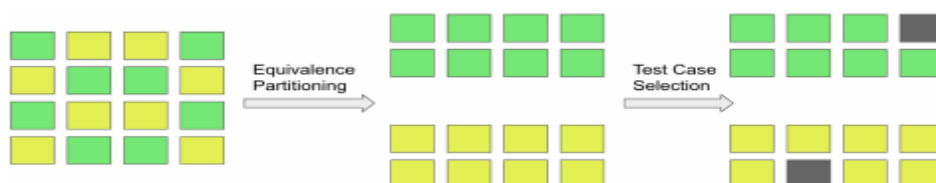
- Valid Input Class = Keeps all valid inputs.
- Invalid Input Class = Keeps all Invalid inputs.

An input has certain ranges which are valid and other ranges which are invalid. Invalid data here does not mean that the data is incorrect; it means that this data lies outside of specific partition. This may be best explained by the example of a function which takes a parameter "month". The valid range for the month is 1 to 12, representing January to December. This valid range is called a partition. In this example there are two further partitions of invalid ranges. The first invalid partition would be ≤ 0 and the second invalid partition would be ≥ 13 .

Steps:

- Divide the input domain into classes of data for which test cases can be generated.
- Attempting to uncover classes of errors.
- Derives test cases based on these partitions
- An equivalence class is a set of valid or invalid states of input
- Test case design is based on equivalence classes for an input domain.

ECP process:



Example1:

- A text field permits only numeric characters
 - Length must be 6-10 characters long
- Partition according to the requirement should be like this:

0 1 2 3 4 5 | 6 7 8 9 10 | 11 12 13 14
 Invalid | Valid | Invalid

- While evaluating Equivalence partitioning, values in all partitions are equivalent that's why 0-5 are equivalent, 6 – 10 are equivalent and 11- 14 are equivalent.
- At the time of testing, test 4 and 12 as invalid values and 7 as valid one.
- It is easy to test input ranges 6–10 but harder to test input ranges 2-600. Testing will be easy in the case of lesser test cases but you should be very careful. Assuming, valid input is 7. That means, you believe that the developer coded the correct valid range (6-10).

Example2:

Check age is between 18 and 60

Invalid	Valid	Invalid
< 18	18 to 60	> 60

Example3:

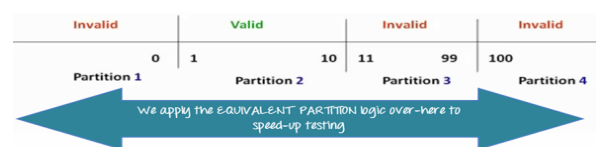
- Let's consider the behavior of Order Pizza Text Box Below
- Pizza values 1 to 10 is considered valid. A success message is shown.
- While value 11 to 99 are considered invalid for order and an error message will appear, **"Only 10 Pizza can be ordered"**

Order Pizza:

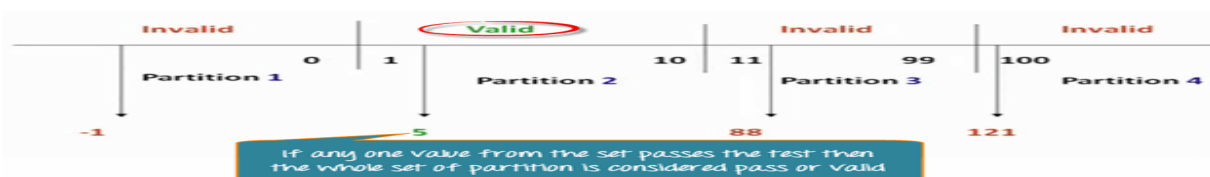
Here is the test condition

1. Any Number greater than 10 entered in the Order Pizza field(let say 11) is considered invalid.
2. Any Number less than 1 that is 0 or below, then it is considered invalid.
3. Numbers 1 to 10 are considered valid
4. Any 3 Digit Number say -100 is invalid.

We cannot test all the possible values because if done, the number of test cases will be more than 100. To address this problem, we use equivalence partitioning hypothesis where we divide the possible values of tickets into groups or sets as shown below where the system behavior can be considered the same.



The divided sets are called Equivalence Partitions or Equivalence Classes. Then we pick only one value from each partition for testing. The hypothesis behind this technique is **that if one condition/value in a partition passes all others will also pass**. Likewise, **if one condition in a partition fails, all other conditions in that partition will fail**.



Example4: OTP Number = 6 digits

INVALID	INVALID	VALID	VALID
1 Test case	2 Test case	3 Test case	
DIGITS >=7	DIGITS <=5	DIGITS = 6	DIGITS = 6
93847262	9845	456234	451483

Advantages	Disadvantages
It is process-oriented	All necessary inputs may not cover.
We can achieve the Minimum test coverage	This technique will not consider the condition for boundary value analysis.
It helps to decrease the general test execution time and also reduce the set of test data.	The test engineer might assume that the output for all data set is right, which leads to the problem during the testing process.

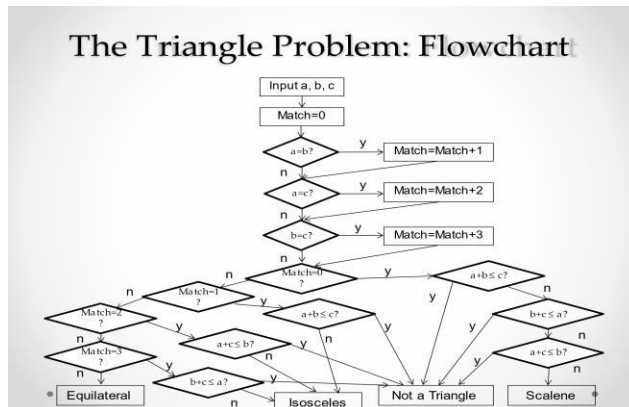
EXERCISE: Consider Triangle problem : /* Design and develop a program in a language of your choice to solve the triangle problem defined as follows : Accept three integers which are supposed to be the three sides of triangle and determine if the three values represent an equilateral triangle, isosceles triangle, scalene triangle, or they do not form a triangle at all. Derive test cases for your program based on ECP approach, execute the test cases and discuss the results */

Test Case Name :Equivalence class Analysis for triangle problem

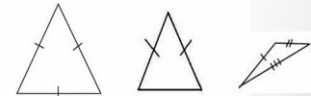
Test Data : Enter the 3 Integer Value(a , b And c)

Pre-condition : $1 \leq a \leq 10$, $1 \leq b \leq 10$ and $1 \leq c \leq 10$ and $a < b + c$, $b < a + c$ and $c < a + b$

Brief Description : Check whether given value for a Equilateral, Isosceles , Scalene triangle or can't form a triangle



- Problem statements (Improved version)
 - input 3 integers: a, b, care side of triangle
- Spec and condition
 - c1: $1 \leq a \leq 10$ C4 $b+c > a$ (As a triangle)
 - c2: $1 \leq b \leq 10$ C5 $a+c > b$ (As a triangle)
 - c3: $1 \leq c \leq 10$ C6 $a+b > c$ (As a triangle)
- Output is type of triangle
 - Equilateral
 - Isosceles
 - Scalene
 - Not a Triangle



Project Name:	Triangle Problem Decision (Equivalence Partitioning)
Test ID:	Solve_Triangle_EP_TESEXX
Test Title:	Use Equivalence partitioning approach for triangle problem
Test Priority:	Low
Module Name:	Identify_Triangle
Test Data:	Enter the 3 Integer Value (a, b and c)
Designed By:	
Designed Date:	
Executed By:	
Executed Date:	
Description of Test:	Check whether the given values form an equilateral, isosceles, scalene triangle or no triangle using equivalence Partitioning approach.
PREREQUISITES:	

Pre-Conditions:	Range in which testing is to be done: $1 \leq a \leq 10$, $1 \leq b \leq$
	10 and $1 \leq c \leq 10$ and $a < b + c$, $b < a + c$ and $c < a + b$
Dependencies:	No dependencies in this test.

Test Case Template:

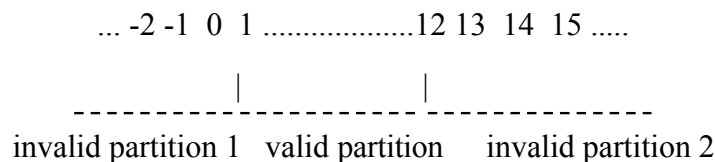
Use test summary report template //attach all screenshots here

Lab No. 4b: Write test cases Using Boundary Value Analysis

Boundary value analysis is software testing design technique in which tests are designed to include representatives of boundary values. Values on the edge of an equivalence partition or at the smallest value on either side of an edge are taken for testing. The values could be either input or output ranges of a software component. Since these boundaries are common locations for errors that result in software faults they are frequently exercised in test cases. It is a Black Box Testing Method, complements to Equivalence partition and BVA leads to a selection of test cases that exercise bounding values.

The expected input and output values should be extracted from the component specification. The input and output values to the software component are then grouped into sets with identifiable boundaries. Each set, or partition, contains values that are expected to be processed by the component in the same way.

For example, if the input values were months of the year expressed as integers, the input parameter 'month' might have the following partitions:

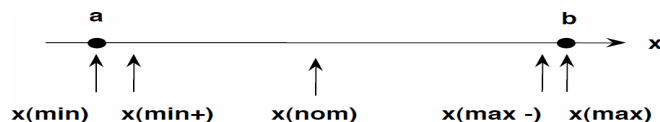


The boundaries are the values on and around the beginning and end of a partition. If possible test cases should be created to generate inputs or outputs that will fall on and to either side of each boundary. This would result in three cases per boundary. The test cases on each side of a boundary should be in the smallest increment possible for the component under test. In the example above there are boundary values at 0,1,2 and 11,12,13.

Boundary testing is the process of testing between extreme ends or boundaries between partitions of the input values.

- So these extreme ends like Start- End, Lower- Upper, Maximum-Minimum, Just Inside-Just Outside values are called boundary values and the testing is called "boundary testing".
- The basic idea in normal boundary value testing is to select input variable values at their:

1. Minimum
2. Just above the minimum
3. A nominal value
4. Just below the maximum
5. Maximum



Example:

- Let's consider the behavior of Order Pizza Text Box Below
- Pizza values 1 to 10 is considered valid. A success message is shown.
- While value 11 to 99 are considered invalid for order and an error message will appear, **"Only 10 Pizza can be ordered"**

Order Pizza:

Submit

Example : Input Box should accept the Number 1 to 10; Here we will see the Boundary Value Test Cases

Test Scenario Description	Expected Outcome
Boundary Value = 0	System should NOT accept
Boundary Value = 1	System should accept
Boundary Value = 2	System should accept
Boundary Value = 9	System should accept
Boundary Value = 10	System should accept
Boundary Value = 11	System should NOT accept

Why Boundary Analysis Testing is important?

1. This testing is used to reduce a very large number of test cases to manageable chunks.
2. Very clear guidelines on determining test cases without compromising on the effectiveness of testing.
3. Appropriate for calculation-intensive applications with a large number of variables/inputs

Disadvantages of boundary value analysis

1. The success of the testing using this technique depends on the equivalence classes identified, which further depends on the expertise of the tester and his knowledge of the application. Hence, incorrect identification of equivalence classes leads to incorrect boundary value testing.
2. Applications with open boundaries or applications not having one-dimensional boundaries are not suitable for this technique. In those cases, other black-box techniques are used.

EXERCISE: recall the triangle problem and write test cases using BVA method using template

Project Name: The Triangle Problem: Flowchart	Triangle Problem Decision (Bound Analysis Approach)
Test Case ID:	Solve_Triangle_TESTXXX(XX=Roman)
Test Title:	Use bound value analysis approach for triangle problem
Test Priority:	Low
Module Name:	Identify Triangle
Test Data:	Enter the 3 Integer Value (a, b and c)
Designed By:	
Designed Date:	
Executed By:	
Executed Date:	
Description of Test:	Check whether the given values form an equilateral, isosceles, scalene triangle or no triangle using bound value analysis approach.
PREREQUISITES:	

Pre-Conditions:	Range in which testing is to be done: $1 \leq a \leq 10$, $1 \leq b \leq$
	10 and $1 \leq c \leq 10$ and $a < b + c$, $b < a + c$ and $c < a + b$
Dependencies:	No dependencies in this test.

Test Case Template:

Use test summary report template //attach all screenshots here

Lab 05: Black Box Testing: Decision Table

The decision table is a software testing technique which is used for testing the system behavior for different input combinations. This is a systematic approach where the different input combinations and their corresponding system behavior are captured in a tabular form. This table helps you deal with different combination inputs with their associated outputs. Also, it is known as the cause-effect table because of an associated logical diagramming technique called cause-effect graphing that is basically used to derive the decision table. It is a black box test design technique use to determine the test scenarios for complex business logic.

A decision table has 4 portions:

1. Condition stub:
2. Condition entry
3. Action stubs
4. Action entry

The **condition stub** part lists the individual inputs upon which the decision depends, while the **action stub** part lists the alternative actions that may be taken. The entry parts then show the conditions under which each action is selected. This is done by arranging the **condition entry** part into columns, where each column specifies some condition on each of the input values, and then placing a cross in the same column of the **action entry** part to indicate the particular action to be taken. All the conditions of the column must be satisfied in order for the column to be selected.

Decision Table Terminology

		Stub		Entry					
Indicate inputs	Conditions	c1		True		False			
		c2		True	False			True	False
		c3		T	F	--		T	F
Indicate outputs	Actions	a1		X	X			X	
		a2		X				X	X
		a3			X			X	
		a4				X			X

↑
Rule

Rules are interpreted as test cases.

Decision Table Terminologies:

- The left most column is the stub portion
- The right is the entry portion
- Condition portion is denoted by c's.
- Action portion is denoted by a's
- A column in entry portion is a rule.
- "don't care" entry :The don't care entry has two interpretation:
 - Condition is irrelevant
 - Condition does not apply Also denoted as "n/a"

Decision Table Types:

Decision tables in which all the conditions are binary(true/false, Yes/No, 0/1) are called Limited Entry Decision Tables. For Limited Entry Decision Tables, if n conditions exist, there must be 2^n rules.

If conditions are allowed to have several values, the resulting tables are called Extended Entry Decision Tables.

Developing Decision Tables

Every decision should be given a name and the logic of the decision table is independent of the sequence in which condition rules are written but the action takes place in the order in which events occur. Wherever possible, duplication of terms and meaning should be avoided and only the standardized language must be used.

The steps of building the concerned tables are given below.

1. Firstly figure out the most essential factors to be considered in making a decision. This will identify the conditions involved in the decision. Only those conditions should be selected which have the potential to either occur or not but partial occurrences are not permissible.
2. Determine the most possible steps that can take place under varying conditions and not just under current condition. This step will identify the actions.
3. Calculate all the possible combinations of conditions. For every N number of conditions there are $2 \times 2 \times 2 \dots$ (N times) combinations to be considered.
4. Fill the decision rules in the table. Entries in a decision table are filled as Y/N and action entries are generally marked as "X". For the conditions that are immaterial a hyphen "-" is generally put. Decision table is further simplified by eliminating and consolidating certain rules. Impossible rules are eliminated. There are certain conditions whose values do not affect the decision and always result in the same action. These rules can be consolidated into a single rule.

Example1: scenario for an ATM where a decision table would be of use.

Conditions
Withdrawal Amount <= Balance?
Credit Granted?
Actions
Withdrawal granted

A customer requests a cash withdrawal. One of the business rules for the ATM is that the ATM machine pays out the amount if the customer has sufficient funds in their account or if the customer has the credit granted. Already, this simple example of a business rule is quite complicated to describe in text. A decision table makes the same requirements clearer to understand:

• **Step 1 – Analyze the requirement and create the first column**

Requirement: “Withdrawal is granted if requested amount is covered by the balance or if the customer is granted credit to cover the withdrawal amount”.

Conditions	Rule1	Rule2	Rule3	Rule4
------------	-------	-------	-------	-------

Withdrawal Amount <= Balance?	T	T	F	F
Credit Granted?	T	F	T	F
Action count	1	1	1	1
Withdrawal granted				

Step 2 – Add column

The number of columns depends on the number of conditions and the number of alternatives for each condition. If there are two conditions and each condition can be either true or false, you need 4 columns. If there are three conditions there will be 8 columns and so on.

Mathematically, the number of columns is $2^{\text{conditions}}$. In this case $2^2 = 4$ columns. Now is the time to fill in the T (TRUE) and F (FALSE) for the conditions.

The simplest is to say that it should look like this:

Step 3: Determine actions

Conditions	Rule1	Rule2	Rule3	Rule4
Withdrawal Amount <= Balance?	T	T	F	F
Credit Granted?	T	F	T	F
Action count	1	1	1	1
Withdrawal granted	T	T	T	F

Enter actions for each column in the table. You will be able to find this information in the requirement. Enter actions for each column in the table. You will be able to find this information in the requirement. Name the columns (the rules). They may be named R1/Rule 1, R2/Rule 2 and so on, but you can also give them more descriptive names.

Step 4 – Reduce the table

Mark insignificant values with “-”. If the requested amount is less than or equal to the account balance it does not matter if credit is granted. In the next step, you can delete the columns that have become identical.

Check for invalid combinations. Invalid combinations are those that cannot happen, for example, that someone is both an infant and senior. Mark them somehow, e.g. with “X”. In this example,

there are no invalid combinations. Finish by removing duplicate columns. In this case, the first and third column are equal, therefore one of them is removed.

Conditions	Rule1	Rule2	Rule3	Rule4
Withdrawal Amount <= Balance?	T	T	F	F
Credit Granted?	T	F	T	F
Action count	1	1	1	1
Withdrawal granted	T	T	T	F
Conditions	Rule1,2	Rule3	Rule4	
Withdrawal Amount <= Balance?	T	F	F	
Credit Granted?	-	T	F	
Action count	2	1	1	
Withdrawal granted	T	F	F	

Test Case ID	Balance	Withdrawal Amount	Credit Granted	Expected Results
1	200	200	Yes	withdrawal granted.
2	100	200	Yes	withdrawal granted.
3	100	200	No	Withdrawal denied

Step 5 – Write Test cases

Write test cases based on the table. At least one test case per column gives full coverage of all business rules.

- Test case for R1: balance = 200, requested withdrawal = 200. Expected result: withdrawal granted.
- Test case for R2: balance = 100, requested withdrawal = 200, credit granted. Expected result: withdrawal granted.
- Test case for R3: balance = 100, requested withdrawal = 200, no credit. Expected Result: withdrawal denied.

Example3: face book login

The four screenshots illustrate the application of decision table testing to the Facebook login process. Each screenshot shows a table of conditions and expected results, with callouts pointing to specific parts of the interface.

Screenshot 1 (Top Left): Shows a table with conditions 'Email or Phone' (F, T) and 'Password' (F, T). The expected results are 'Error: Incorrect Email' and 'Error: Please Confirm Password'. A callout points to the 'Please Confirm Password' message on the login page.

Screenshot 2 (Top Right): Shows a table with conditions 'Email or Phone' (F, T, F, T) and 'Password' (F, F, T, T). The expected results are 'Error: Incorrect Email', 'Error: Please Confirm Password', 'Error: Incorrect Email', and 'Facebook Page is Processed'. A callout points to the 'Facebook Page is Processed' message on the login page.

Screenshot 3 (Bottom Left): Shows a table with conditions 'Email or Phone' (F, T) and 'Password' (F, T). The expected results are 'Error: Incorrect Email', 'Error: Please Confirm Password', 'Error: Incorrect Email', and 'Facebook Page is Processed'. A callout points to the 'Condition Entry' label.

Screenshot 4 (Bottom Right): Shows a table with conditions 'Email or Phone' (F, T) and 'Password' (F, T). The expected results are 'Error: Incorrect Email', 'Error: Please Confirm Password', 'Error: Incorrect Email', and 'Facebook Page is Processed'. A callout points to the 'Action Stubs' label.

Screenshot 5 (Bottom): Shows a table with conditions 'Email or Phone' (F, T) and 'Password' (F, T). The expected results are 'Error: Incorrect Email', 'Error: Please Confirm Password', 'Error: Incorrect Email', and 'Facebook Page is Processed'. A callout points to the 'Action Entry' label.

Table 1 (Screenshot 1):

Condition	Rule1	Rule2
Email or Phone	F	T
Password	F	F
Expected Results	Error: Incorrect Email	Error: Please Confirm Password

Table 2 (Screenshot 2):

Condition	Rule1	Rule2	Rule3	Rule4
Email or Phone	F	T	F	T
Password	F	F	T	T
Expected Results	Error: Incorrect Email	Error: Please Confirm Password	Error: Incorrect Email	Facebook Page is Processed

Table 3 (Screenshot 3):

Condition	Rule1	Rule2	Rule3	Rule4
Email or Phone	F	T	F	T
Password	F	F	T	T
Expected Results	Error: Incorrect Email	Error: Please Confirm Password	Error: Incorrect Email	Facebook Page is Processed

Table 4 (Screenshot 4):

Condition	Rule1	Rule2	Rule3	Rule4
Email or Phone	F	T	F	T
Password	F	F	T	T
Expected Results	Error: Incorrect Email	Error: Please Confirm Password	Error: Incorrect Email	Facebook Page is Processed

Table 5 (Screenshot 5):

Condition	Rule1	Rule2	Rule3	Rule4
Email or Phone	F	T	F	T
Password	F	F	T	T
Expected Results	Error: Incorrect Email	Error: Please Confirm Password	Error: Incorrect Email	Facebook Page is Processed

A rule = A test case

No. of combination = No. of Condition1 value * No. of condition2 value
= 2 * 2 = 4 rules or 4 test cases

Why is Decision Table Important?

A decision table is an outstanding technique used for testing and requirements management. Some of the reasons why the decision table is important include:

- Decision tables are very much helpful in test design technique.
- It helps testers to search the effects of combinations of different inputs and other software states that implement business rules.
- It provides a regular way of stating complex business rules which benefits the developers as well as the testers.
- It assists in the development process with the developer to do a better job. Testing with all combination might be impractical.
- It is the most preferable choice for testing and requirements management.
- It is a structured exercise to prepare requirements when dealing with complex business rules.
- It is also used in model complicated logic.

EXERCISE:

1. Consider Triangle problem : /* Design and develop a program in a language of your choice to solve the triangle problem defined as follows : Accept three integers which are supposed to be the three sides of triangle and determine if the three values represent an equilateral triangle, isosceles triangle, scalene triangle, or they do not form a triangle at all. Derive test cases for your program based on decision-table approach, execute the test cases and discuss the results */ and attached all printouts /screenshots/summary report here

Conditions:

- $a < b + c?$
 - $b < a + c?$
 - $c < a + b?$
 - $a = b?$
 - $a = c?$
 - $b = c?$
- } **a, b, c, form a triangle?**

Actions:

- Not a Triangle
- Scalene
- Isosceles
- Equilateral
- Impossible

- ii. Make initial decision table consisting of columns (using the formula $2^{\text{conditions}}$)
- iii. Reduced the table to remove redundant/impossible scenarios.
- iv. write test cases for reduced decision table
- v. execute code for triangle problem and execute test cases (iii)

2. Consider a library management system(LMS) which issues book(s) to a registered user by checking the outstanding fee i.e. if a registered user have no pending fee then requested book may be issued(if and only if the potential user has under borrow limit)

- i. Make initial decision table consisting of columns (using the formula $2^{\text{conditions}}$)
- ii. Reduced the table to remove redundant/impossible scenarios.
- iii. write test cases for reduced decision table
- iv. execute code for LMS problem and execute test cases (iii)

Design and develop a program in a language of your choice to develop above mentioned problem and attached all printouts /screenshots/summary report here.

Lab 06a

Exercise: Select your previous project and Design Test cases for at least 5 components using any of white box strategy also answer the following questions

- i. Draw a comparison of selected techniques with all other ones.**
- ii. Why you choose this technique?**
- iii. On what level we can apply white box and when will we should stop software testing**

Attached all printouts and screenshots of application [code+ application o/p on test data] here. Also attached summary report and calculate code coverage of each test case.

Lab 06b

Exercise: Select your previous project and Design Test cases for at least 5 features using any of black box strategy [you can choose different techniques to test different features] also answer the following questions

- i. Draw a comparison of selected techniques with all other ones.**
- ii. Why you choose this technique?**
- iii. On what level we can apply black box and why?**

Attached all printouts and screenshots of application [UI+ application o/p on test data] here. Also attached summary report.

Lab 7: Unit Testing in Python

Unit testing is a crucial aspect of software development that involves testing individual units or components of code to ensure their proper functioning. It aims to validate that each unit, such as a function or method, works as intended and produces the expected output.

Python, being a popular programming language, offers several frameworks and tools to facilitate unit testing. The Python unit testing process typically follows a set of steps.

First, developers write test cases to validate the behavior of a unit. Test cases often cover both routine and edge cases to ensure robustness.

Next, developers utilize a unit testing framework, such as PyTest, unittest, or doctest, to organize and execute these test cases.

The framework provides essential features like assertion methods to check the expected output against the actual output.

What is the Python unittest module?

The unittest module is a built-in testing framework in Python, available as part of the standard library. The primary goal of the unittest module is to facilitate the creation of test cases and test suites to verify the behavior and correctness of individual units of code.

- These units are typically functions, methods, or classes, and the purpose of unit testing is to ensure that each unit functions as intended and produces the expected output.
- The unittest module offers a class-based approach to define test cases. Developers create a test case class that inherits from the `unittest.TestCase` class.
- Within this test case class, individual test methods are defined, where each method represents a specific test scenario.
- Test methods are named starting with the prefix “test_” to be discovered by the testing framework.

Key Features of Unittest:

The unittest module in Python offers several benefits for writing and executing unit tests. Some of the key advantages of using unittest for testing your code include:

- **Built-in Testing Framework:** unittest is part of Python’s standard library, which means it is readily available without the need for additional installations. This makes it easily accessible and ensures compatibility across different Python environments.
- **Simple and Familiar Syntax:** The unittest framework utilizes a class-based approach, where test cases are defined as classes inheriting from **unittest.TestCase** which is intuitive and familiar to developers.
- **Assertion Methods:** unittest provides a wide range of assertion methods to compare expected and actual results.
- **Test Discovery:** The unittest framework supports automated test discovery, which enables the discovery and execution of all test cases within a directory or module.
- **Test Fixtures:** With unittest, you can define setup and teardown methods within test cases. These fixture methods, such as **setUp()** and **tearDown()**, allow you to set up the necessary environment before executing each test and clean up any resources afterward.
- **Test Suites:** unittest supports the creation of test suites, allowing you to group related test cases together.
- **Integration with Test Runners:** The unittest module integrates well with test runners, which are responsible for discovering and executing tests.

- **Extensibility:** The unittest framework is designed to be extensible. Developers can subclass **unittest.TestCase** to create custom test case classes with additional functionality or override existing methods.
- **Compatibility and Community Support:** Being a part of Python's standard library, unittest enjoys widespread usage and community support. It is well-documented, and developers can find numerous online resources, tutorials, and examples to guide them in utilizing the framework effectively.

Prerequisites for Setting up Python Unit Testing

To set up Python unit testing, there are a few prerequisites you should consider:

1. **Python Installation:** Ensure that Python is installed on your system. You can download the latest version of Python from the official Python website (<https://www.python.org>) and follow the installation instructions for your operating system.
2. **Testing Framework:** Choose a testing framework that suits your needs. Python offers several options, including **unittest**, **PyTest**, and **doctest**.
3. **Project Setup:** Set up your project structure. Create a dedicated directory for your project, and organize your source code files, modules, and tests accordingly. A well-structured project makes it easier to manage and run unit tests.
4. **Testing Dependencies:** You can typically install dependencies using **Python's package manager, pip**, by executing commands like **pip install <package-name>**.
5. **Virtual Environments:** Virtual environments allow you to have separate Python environments for different projects, avoiding conflicts between packages and ensuring project-specific dependencies are installed.
6. **Editor or IDE:** Choose a code editor or IDE for writing your tests. Popular options include **Visual Studio Code**, **PyCharm**, **Atom**, and **Sublime Text**. An editor or IDE with built-in support for Python and unit testing can provide helpful features like syntax highlighting, code completion, and test execution within the development environment.
7. **Test Runner:** Familiarize yourself with the concept of test runners. A test runner is a tool or command-line utility that discovers and executes unit tests. Many testing frameworks, including **unittest** and **PyTest**, come with built-in test runners. Understand how to use the test runner of your chosen framework to run your unit tests effectively.
8. **Test Coverage:** Consider using a code coverage tool to measure the extent to which your unit tests cover your codebase. Tools like **coverage.py** can help you identify areas of your code that lack test coverage, ensuring that your tests are comprehensive and thorough.

How to Define Unit Test Cases for Python Functions?

Defining test cases for Python functions involves creating test methods that verify the behavior of the function under various scenarios. Each test method represents a specific test scenario, such as testing different input values, edge cases, or expected error conditions. Assertions are used within the test methods to compare the expected output of the function with the actual output.

To define test cases for Python functions, You can follow the below steps:

1. **Import the unittest Module:** Start by importing the unittest module at the beginning of your test file.

```
import unittest
```
2. **Create a Test Case Class:** Define a test case class that inherits from **unittest.TestCase**. This class will contain your test methods.

```
class MyTestCase(unittest.TestCase):
```
3. **Define Test Methods:** Within the test case class, define individual test methods. Each test method should start with the prefix **test_** to be discovered by the testing framework.

```
class MyTestCase(unittest.TestCase):
```

```
    def test_function_name(self):
```

```
        # Test code goes here
```

4. Write Assertions: Inside each test method, write assertions to check the expected behavior of the function being tested. Assertions compare the actual output of the function to the expected output.

```
class MyTestCase(unittest.TestCase):
```

```
    def test_function_name(self):
```

```
        result = my_function() # Call the function being tested
```

```
        self.assertEqual(result, expected_result)
```

You can use various assertion methods provided by unittest like `assertEqual`, `assertTrue`, `assertFalse`, etc., depending on the specific conditions you want to check.

5. Run the Tests: To execute the test cases, you need to run a test runner. You can use the **unittest test runner** provided by Python. You can either run the tests from the command line or use a testing framework or IDE that supports test execution.

From the command line, execute the following command in the same directory as your test file:

```
python -m unittest <test_file.py>
```

Alternatively, you can use a testing framework or IDE that provides a graphical interface or test runner integration for executing the tests.

Here's an example that demonstrates how to define test cases for a simple Python function

```
def multiply(a, b):  
    return a * b
```

Let's define test cases for the multiply function using the unittest module:

```
import unittest
```

```
class MultiplyTestCase(unittest.TestCase):
```

```
    def test_multiply_positive_numbers(self):
```

```
        result = multiply(3, 4)
```

```
        self.assertEqual(result, 12)
```

```
    def test_multiply_negative_numbers(self):
```

```
        result = multiply(-2, -5)
```

```
        self.assertEqual(result, 10)
```

```
    def test_multiply_zero(self):
```

```
        result = multiply(10, 0)
```

```
        self.assertEqual(result, 0)
```

```
    def test_multiply_with_one(self):
```

```
        result = multiply(7, 1)
```

```
        self.assertEqual(result, 7)
```

- In this example, we define a test case class **MultiplyTestCase** that inherits from **unittest.TestCase**. Inside this class, we define individual test methods, each starting with the prefix **test_**.
- In the **test_multiply_positive_numbers** method, we test the **multiply function** with positive numbers 3 and 4. We expect the result to be 12, so we use **self.assertEqual(result, 12)** to check if the actual result matches the expected result.
- In the **test_multiply_negative_numbers** method, we test the **multiply function** with negative numbers -2 and -5. We expect the result to be 10, so we use **self.assertEqual(result, 10)** for assertion.
- In the **test_multiply_zero** method, we test the **multiply function** with one operand as 0. We expect the result to be 0, so we use **self.assertEqual(result, 0)** for assertion.
- In the **test_multiply_with_one** method, we test the multiply function with one operand as 1. We expect the result to be the other operand itself, so we use **self.assertEqual(result, 7)** for assertion.

And if required to run with a specific file then give the file name in command like this **python -m unittest -v <filename>.py**.

```
===== test session starts =====
collecting ... collected 4 items

s.py::MultiplyTestCase::test_multiply_negative_numbers PASSED [ 25%]
s.py::MultiplyTestCase::test_multiply_positive_numbers PASSED [ 50%]
s.py::MultiplyTestCase::test_multiply_with_one PASSED [ 75%]
s.py::MultiplyTestCase::test_multiply_zero PASSED [100%]

===== 4 passed in 0.04s =====

Process finished with exit code 0
```

Assert Methods for Unit Testing in Python

When writing unit tests for Python functions using the **unittest module**, you can use various assertion methods provided by the module to compare expected and actual values.

Here are 10 commonly used assertion methods for Python functions using the unittest module:

- 1. assertEquals(a, b):** Checks if a and b are **equal**.
self.assertEqual(10, add(5, 5)) # Passes if add(5, 5) equals 10
- 2. assertTrue(x):** Checks if x evaluates to **True**.
self.assertTrue(result) # Passes if result is True
- 3. assertFalse(x):** Checks if x evaluates to **False**.
self.assertFalse(error) # Passes if error is False
- 4. assertIs(a, b):** Checks if a is the **same object** as b.
self.assertIs(result, expected_result) # Passes if result is expected_result (same object)
- 5. assertIsNone(x):** Checks if x is **None**.
self.assertIsNone(result) # Passes if result is None
- 6. assertIsNotNone(x):** Checks if x is **not None**.
self.assertIsNotNone(result) # Passes if result is not None
- 7. assertIn(a, b):** Checks if a is **present** in b.
self.assertIn(item, my_list) # Passes if item is present in my_list
- 8. assertNotIn(a, b):** Checks if a is **not present** in b.
self.assertNotIn(item, my_list) # Passes if item is not present in my_list
- 9. assertRaises(exception, callable, *args, **kwargs):** Checks if calling **callable** raises **exception**.
self.assertRaises(ValueError, divide, 10, 0) # Passes if calling divide(10, 0) raises ValueError

10. assertAlmostEqual(a, b, places): Checks if **a** and **b** are approximately **equal up to a** specified number of decimal places.

```
self.assertAlmostEqual(result, expected_result, places=2) # Passes if result and expected_result are approximately equal up to 2 decimal places
```

These are just a few examples of commonly used assertion methods provided by **unittest**. There are more assertion methods available, including those for checking exception messages, container equality, string containment, and more. Choose the appropriate assertion method based on the specific conditions you want to test and the expected behavior of your function.

Exercise:

1. Validate Basic Calculator application through unittest module. use at least 6 assertions methods in your testfile. //Attach all code snippets and outputs here.
2. Validate any 2 module of your previous python application through unittest //. //Attach all code snippets and outputs here.

3.

Lab# 08 a : Introduction to Katalon Studio + Installation

Katalon Studio is an automation testing solution, developed by Katalon, Inc. The software is built on top of the open-source automation frameworks Selenium, Appium with a specialized IDE interface for web, API, mobile and desktop application testing. Katalon Studio provides a dual interchangeable interface for creating test cases: a manual view for the less technical users and a script view gearing toward experienced testers to author automation tests with syntax highlight and intelligent code completion.

It follows the Page Object Model pattern. GUI elements on web, mobile, and desktop apps can be captured using the recording utility and stored into the Object Repository, which is accessible and reusable across different test cases. Test cases can be structured using test suites with environment variables. Test execution can be parameterized and parallelized using profiles. Remote execution in Katalon Studio can be triggered by CI systems via Docker container or command line interface (CLI). The test automation framework provided within Katalon Studio was developed with the keyword-driven approach as the primary test authoring method with data-driven functionality for test execution. The user interface is a complete integrated development environment (IDE) implemented on Eclipse rich client platform (RCP).

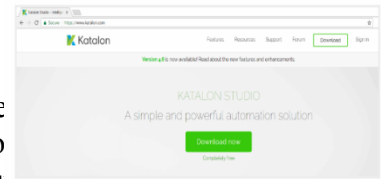
Katalon's main advantage is that it's easy to deploy and has a wider set of integrations compared to Selenium, the market leader. Katalon has dual scripting interfaces for users with different programming skills. This means that testers with limited technical knowledge can use a simpler user interface that doesn't require writing code. The mode for more proficient users has access to scripting with syntax highlighting, code suggestion, and debugging. Katalon Studio has pre-defined artifact structure: a number of templates for test cases, test suites, test objects, and reports. Katalon Studio supports local and remote testing, as well as parallel and sequential executions. It runs on Groovy (Java) scripting language. Katalon Studio is a cross-browser tool that supports Web, mobile, and API testing. These solutions go with analytics and recording modules.

Katalon Studio Key Features

KATALON STUDIO KEY FEATURES	
Testing environments	Windows (7,8,10), macOS (10, 11+), Linux
Applications under test	Web, API, mobile (Android 6.x and 7.x, iOS 9-11)
Scripting languages	Groovy (Java)
Supported browsers	Google Chrome Firefox Internet Explorer Safari Edge Remote Headless browsers + custom environments
Programming skills	Not required
Learning curve	Mild
Script creation time	Quick
Object storage and maintenance	Page-object model
Continuous integration	Various CI and DevOps tools (e.g. Jenkins, Bamboo, TeamCity, Git, JIRA)
Product support	Dedicated staff
Cost	Free (paid support)

API testing

Unlike Selenium and Ranorex, with Katalon Studio you can test integrations. This inbuilt module allows testers to conduct end-to-end scripting, and maintain their tests. Main features of this module are code inspection, code snippeting, references, and debugger. The software supports SOAP/1.1 and SOAP/1.2 requests and multiple data sources (XLS, XML, databases with dynamic mapping to maximize test coverage). The API testing mode allows users to import tests from such API testing and editing tools as Swagger, Postman, and WSDL. Katalon API testing also has an inbuilt response viewer with auto-formatting and search for access to artifacts.



Web testing

Katalon offers a complete web testing solution with inbuilt Continuous Delivery/Continuous Integration and DevOps integrations. It supports testing of Web, Android, and iOS applications. The tool has code-assist utilities like built-in object spy, code refactoring, and in-context references. Native plugins integrate with Jenkins, Git, JIRA, and qTest. Katalon's web-testing execution mechanism uses multiple configurations and datasets. It also allows for customizing execution workflow and running it automatically. The module features importing external libraries to improve the automation functions.

Mobile testing

This module allows users to test mobile Web, iOS, and Android apps. With native integration by means of Appium, a test automation tool for mobile applications, the module supports testing for the latest mobile platforms and devices without additional installations. Katalon speeds up mobile testing by detecting and storing the objects.

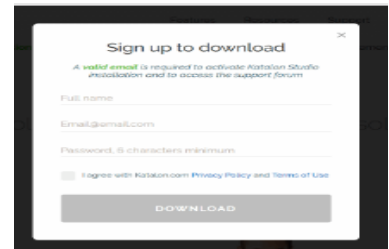
Supported technologies

- Modern web technologies: HTML HTML5, JavaScript, Ajax, Angular
- Windows desktop apps platforms:^[21] Universal Windows Platform (UWP), Windows Forms (WinForms), Windows Presentation Foundation (WPF), and Classic Windows (Win32)
- Cross-browser testing: Firefox, Chrome, Microsoft Edge, Internet Explorer (9,10,11), Safari, headless browsers
- Mobile apps: Android and iOS (Native apps and mobile web apps)
- Web services: RESTful and SOAP

System Requirements:

- ✓ Operating System: Windows 7, Windows 8, Windows 10, macOS 10.11+
- ✓ CPU: 1 GHz or faster 32-bit (x86) or 64-bit (x64) processor
- ✓ Memory: 1 GB RAM (32-bit) or 2 GB RAM (64-bit)
Hard Drive: at least 1 GB available hard disk space

How to download and Install:



Sign up to download

A **valid email** is required to activate Katalon Studio
activation and to access the support forum

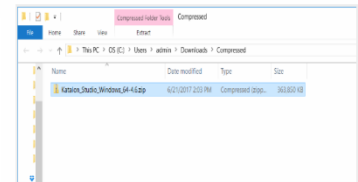
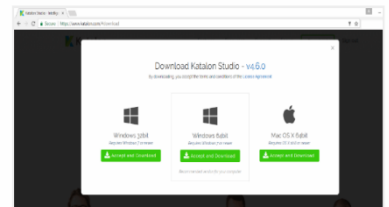
Full name: _____

Email:

Password: 8 characters minimum

☐ I agree with Katalon.com Privacy Policy and Terms of Use

DOWNLOAD



Name	Date modified	Type	Size
Katalon_Studio_Windows_64-6.0.0	6/21/2017 2:53 PM	Compressed zip...	361,803 KB

TestProject [Sign Up >](#)

Product Activation

Email

Password

[Clear](#) [Activate](#) [Forgot Password?](#) | [Register](#) | [Offline Activation](#) | [Config Proxy](#)

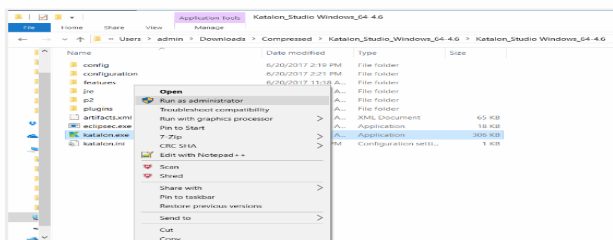
1. Go to Katalon Studio
2. Click on “Download”

3. Give your full name, valid email, password, read and agree to the Katalon Privacy Policy and click on “Download

4. Choose a required download version and click on “Accept and Download”

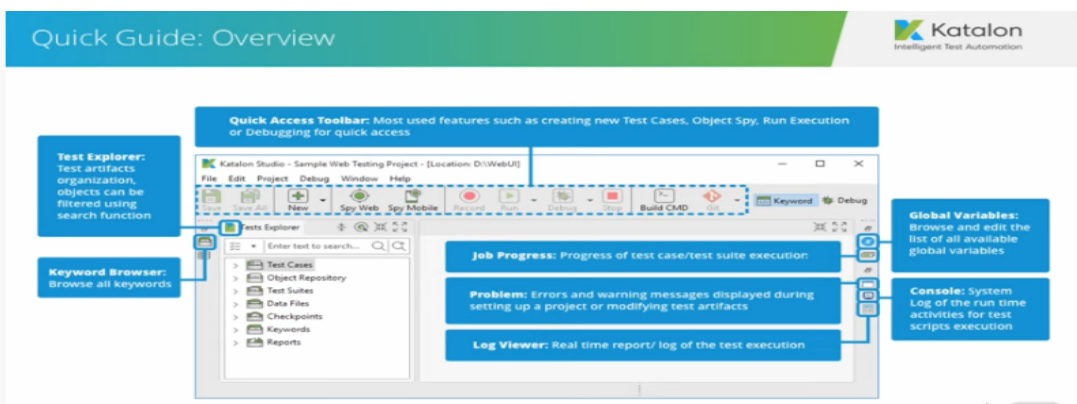
5. Once the download is complete – Go to the folder on your computer where the Katalon Studio is downloaded

6. Run “Katalon.exe” to install Katalon Studio



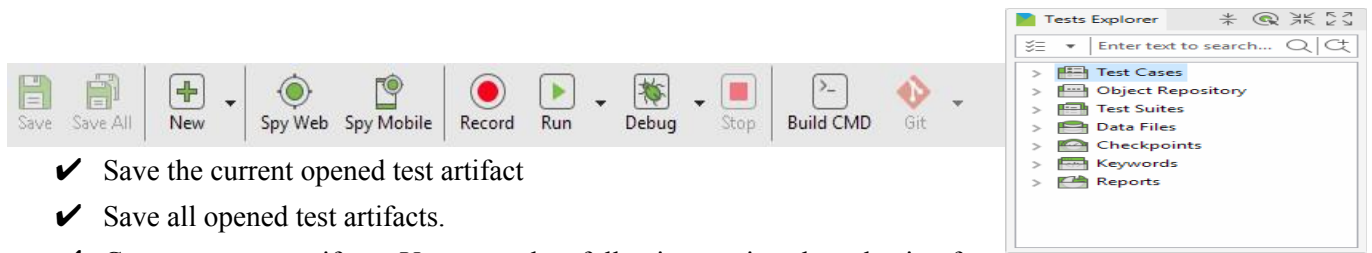
7. Enter your account credentials that you used when signing up

8. Katalon launches and you are ready to automate.



Lab # 8 b: Explore Katalon GUI

Toolbar: The main **Toolbar** contains the most common actions which you usually perform (e.g. creating test resources or executing automation tests).



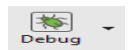
- ✓ Save the current opened test artifact
- ✓ Save all opened test artifacts.
- ✓ Create new test artifacts. You can select following options by selecting from dropdown list:

Folder	Web Service Request
Test Case	Test Data
Test Suite	Checkpoint
Test Suite Collection	Package
Test Object	Keyword



- ✓ Open **Object Spy** dialog for capturing elements on websites.
- ✓ Open **Mobile Object Spy** dialog for capturing elements on mobile applications.
- ✓ Open **Record** dialog for recording WebUI test cases.
- ✓ Open **Mobile Recorder** dialog for recording Mobile test cases.
- ✓ Run the current open test case. You can select these options by selecting from dropdown list:

Chrome	Remote
Firefox	Headless
IE	Android
Safari	iOS (on macOS)
Edge	Custom



- ✓ Stop the current execution
- ✓ Open **Command Builder** for generating commands for console execution
- ✓ Command for Git activities. You can select these options by selecting from dropdown list (after enabling Git):

Tests Explorer View: The Tests Explorer view allows you to browse the structure of your projects and access all test artifacts quickly. Using the context menu on the view, you can create new artifacts, organize the view's items or drag and drop them to certain editor views if needed.

Group	Description
Test Cases	List all test cases within current project
Object Repository	List all test objects of the current project
Test Suites	List all test suites and test suite collections of the current project
Data Files	List all test data of the current project

Checkpoints	List all checkpoints of the current project
Keywords	List all custom keywords of the current project

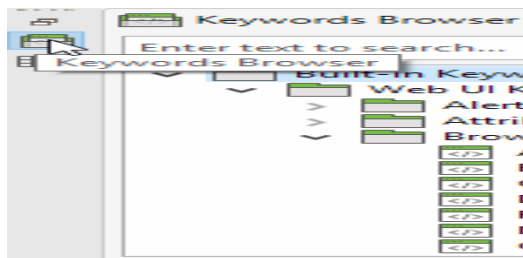
No.	Name	Default value type	Default value
1	Username	String	'John Doe'
2	Password	String	'ThisIsNotAPassword'

Keywords Browser View: displays all available keywords and drop the keywords here to the Test Case editor.

Editors: The editor is used to modify the detailed information of the test case in the editor.

Test Case Editors: When you open a test case, its details are displayed in the Manual tab, Script tab, Variables tab, Integration tab.

✓ **Manual Tab:** The Manual tab displays the Manual editor where the basic keyword-driven configuration is done.



users to create automation tests effortlessly. Refer to the following screenshot.

✓ **Script Tab:** This tab displays the Script view where you can modify test scripts easily using either Groovy or JavaScript.

✓ **Variable Tab:** The Variables tab shows all variables used in the test case. Refer to the Variables tab for more details.

✓ **Integration Tab:** Information regarding the test case integration with qTest is displayed in this tab. Refer to Integrate test case for more details.

Test Case ID	Alias	Parent ID
6633654	TC-205	381730

ID	Created Date	Tag	Description	Comment
Test Cases/Main Test Cases/TC1_Verify Successful Login	Wed Jun 28 16:08:44 ICT 2017		Verify if users can login successful. Steps: - Click on 'Book Appointment' button to login. - Input username and password - Click on 'Login' button Expected result: 'Appointment' section is displayed after successful login	

Name	Match Condition	Value	Detect object by?
id	equals	id("download-btn-banner")	☐
href	equals	#katalon-download	☐
text	equals	Free Download	☐
class	equals	btn btn-default kat-btn-default btn-lg	☒

Name	Value
postid	1

Authorization
No Authorization

No.	ID	Description	Run
1	Test Cases/Main Test Cases/TC1_Verify Successful Login	Verify if users can login	☒
2	../TC2_Verify Successful Appointment	Verify if users can book	☒

- ✓ **Properties Tab:** General information about the test case is displayed in this tab.

Test Object Editor: When you open a test object, its detailed information including properties and object identification mechanism is displayed in the Test Object editor. Refer to Spy Object for more details.

Web Service Editor:

- RESTful Request Object Editor:** When you open a RESTful request object, its detailed information including the resource URL, request methods, parameters... is displayed in the editor. Refer to [Web Services](#) for more details.
- SOAP Request Object Editor:** When you open a SOAP request object, its detailed information including the resource URL, request methods, parameters... is displayed in the editor. Refer to [Web Services](#) for more details.

Test Suite Editor: When you open a test suite, its detailed information is displayed in the Test Suite editor that contains the Main tab, Integration tabs

- ✓ **Main tab:** The main tab displays basic information about the test suite such as the test cases to be executed, the execution mechanism and data binding. Refer to [Main](#) for more details.
- ✓ **Integration tab:** Information regarding your test suite is displayed in the Integration tab. Refer to [Integrate test suite](#) for more details.

Test Suite Collection Editor: a Test Suite Collection contains a set of test suites to allow users to execute test suites together. Refer to [Execute a test suite collection](#) for more details.

Data File Editor: When you open a data file, its detailed information including the data source and preview data set... is displayed in the editor. Refer to [Manage Test Data](#) for more details.

- If the test data is from an Excel file:

The 'New Excel Data' dialog box is shown with the following details:

- File's Information:**
 - File Name: D:\account_list.x
 - Sheet Name: Team A
 - ☒ Use first row as header
 - ☐ Use relative path
- Preview Table:**

No.	Name	Gender	Age
1	user1	M	32
2	user2	M	14
3	user3	F	55

- If the test data is from a CSV file:

The screenshot shows the 'Integration' tab of the Test Suite Editor for 'TS_RegressionTest'. It includes a table for 'List of test suite's parent' and 'Integration Information'.

Name	Type	Default
Release 1 / ...	Cycle	☒

Test Suite ID	686725
Alias	TS-73
Parent ID	140731

Execution Information

No.	ID	Run with	Run Configuration	Run
1	Test Suites/TS_RegressionTest	Chrome	☒	☒
2	Test Suites/TS_RegressionTest	Firefox	☒	☒

New CSV Data

File's Information

File Name: Separator:

☒ Use first row as header ☐ Use relative path

No.	account	country	type
1	acc1	US	free
2	acc2	VN	premium
3	acc3	CN	free

- If the test data is defined internally inside your test project:

New Internal Data

No.	Col_A	Col_B	(+)
1	A1	B1	
2	A2	B2	
(+)			

Checkpoint Editor: When you open a checkpoint, its detailed information including the data source and its taken snapshot... is displayed in the editor. Refer to Manage Checkpoint for more details.

- If the data source is from an Excel file:

***Excel Checkpoint**

Source Info Snapshot was taken on Tue Feb 28 15:46:09 ICT 2017

File Path:

No.	Name	Gender	Age
1	☐ user1	☐ M	☐ 32
2	☐ user2	☐ M	☐ 14
3	☐ user3	☐ F	☐ 55

- If the data source is from a CSV file:

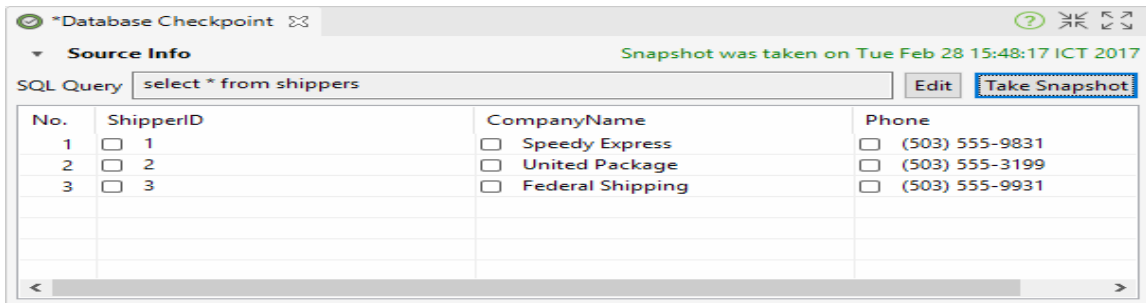
***CSV Checkpoint**

Source Info Snapshot was taken on Tue Feb 28 15:47:06 ICT 2017

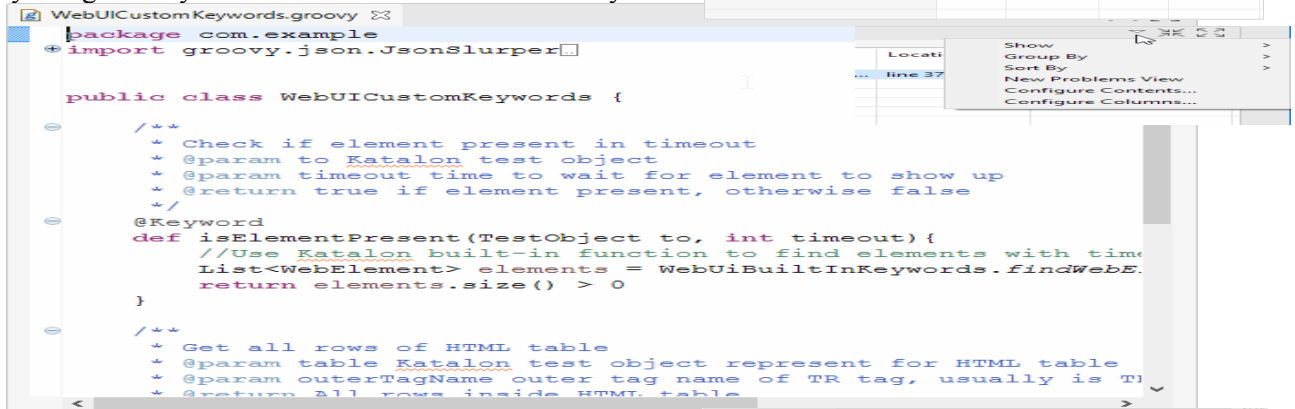
File Path:

No.	account	country	type
1	☐ acc1	☐ US	☐ free
2	☐ acc2	☐ VN	☐ premium
3	☐ acc3	☐ CN	☐ free

- If the data source is from a database:



Keyword Editor: When you open a custom keyword, its content editing script view is similar to the Script view of test cases where you can easily use Groovy or Java. Refer to Define custom keywords

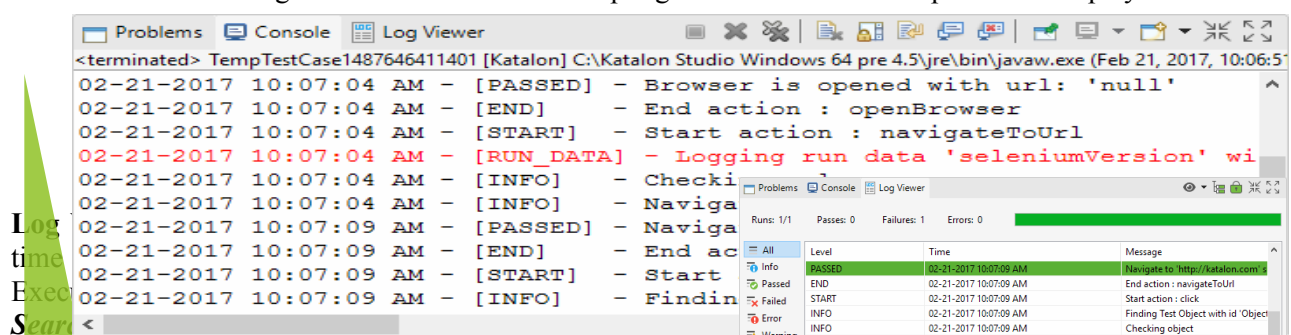


Global Variable View: The Global Variables view allows you to view and manage global variables defined in your project. Refer to Global Variables for

Job Progress View: The Job Progress view allows you to see the progress of executing test cases and test suites.

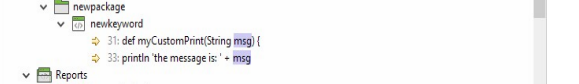
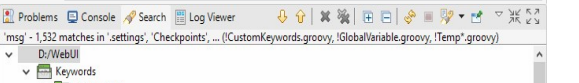
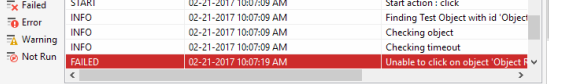
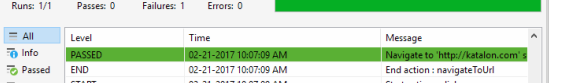
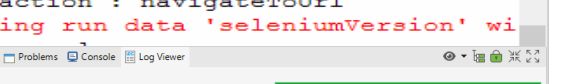
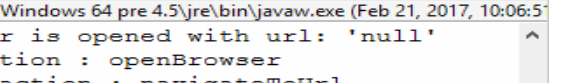
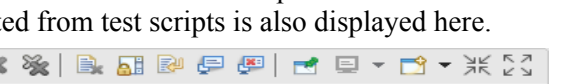
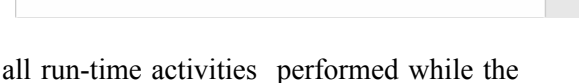
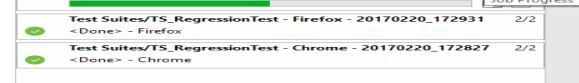
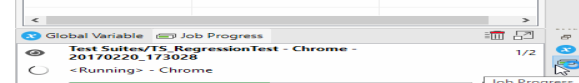
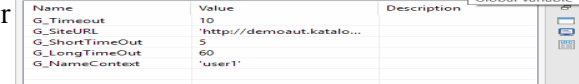
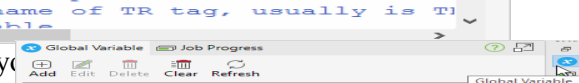
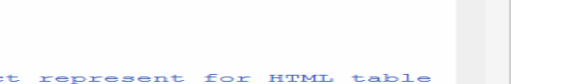
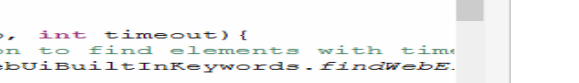
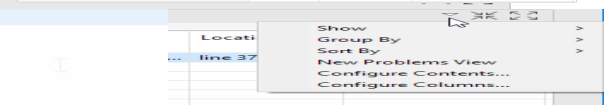
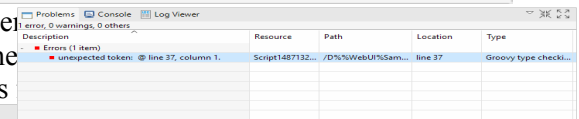
Problems View: The Problems view shows errors and warning messages raised when you are setting up a project or designing the test case, test suite, test object or test data. The messages displayed in the Problems view can be configured via the drop-down menu of this view.

Console View: The Console view shows the system logs of all run-time activities performed while the automation test is being executed. The console output generated from test scripts is also displayed here.



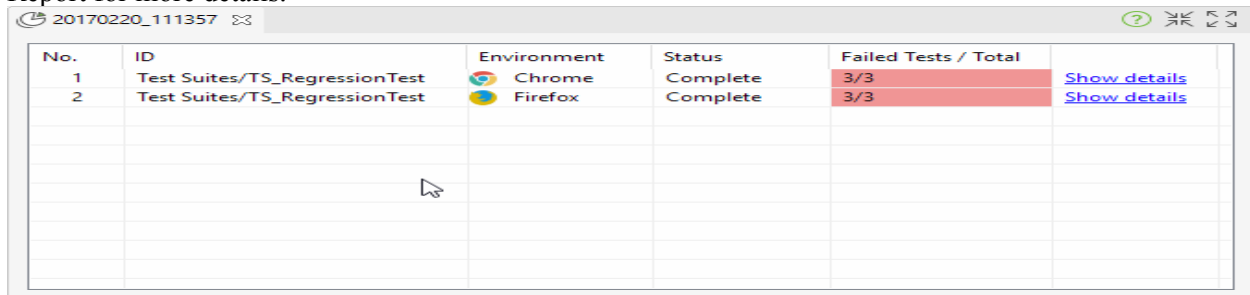
from the search function. You can double-click on a search entry to navigate to the corresponding position in the editor.

Report View: The Report view allows you to view detailed information of a completed test execution for a certain test



suite. Refer to Test Suite Report for more details.

Test Suite Collection Report View: The Test Suite Collection Report view allows you to view detailed information of a completed test execution for a certain test suite collection. Refer to Test Suite Collection Report for more details.



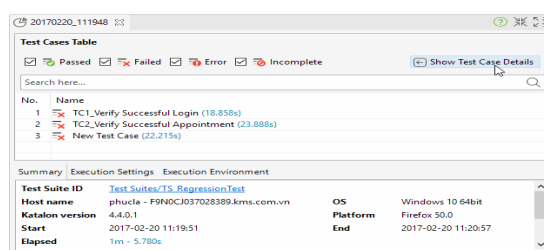
No.	ID	Environment	Status	Failed Tests / Total	
1	Test Suites/TS_RegressionTest	Chrome	Complete	3/3	Show details
2	Test Suites/TS_RegressionTest	Firefox	Complete	3/3	Show details

Who uses Katalon Studio?

- Care Logistics, a company that provides healthcare logistics software and services to hospitals and health centers, faced the challenge of a quick verification of system stability. To verify the system's functionality, the testing team had to create over 8,000 manual test cases and execute all of them before deployment. With Katalon Studio, the team managed to convert and automate these tests. As a result, the team could verify more issues and had 30 percent of products automated. Also, they achieved a 50 percent reduction in regression testing lifecycle.
- Tunaiku, an Indonesian fintech lending venture, had to make sure that user data and transaction information was safe. This required a number of quality assurance tests in CI workflow, which involves Jenkins and Docker integration. Also, they needed to generate automation test scripts effectively. Tunaiku solved these quality assurance team problems by creating automation test scripts and customized code infrastructure in Katalon. This helped them reduce scripting time by 60 percent.
- Angler, an Internet and technologies provider from India, used Katalon Studio for API testing. Their task was to test APIs and generate API scripts, as well as detect bugs in the development phase and prevent them from getting into APIs. As a result, Angler covered over 80 percent of functional testing with Katalon Studio.

Why Choosing Katalon Open Source

- Katalon Studio complexities provides friendly mode (user can keywords and test objects to form test steps), but still keeps necessary weapons for more technically powerful users who are able to dig deeper into coding with the scripting mode that fully supports development conveniences like syntax highlighting, code suggestion and debugging
- With Selenium, users have to go through technical guidelines for their API and integrate them into with an IDE and a preferred programming language. By contrast, Katalon Studio is built as a unified bundle which includes almost all necessary things like Java, Android SDK, Web drivers to drive browsers, and required dependencies. All you have to do is to download and install it on your computer. Minimum installation is required



Studio Over Selenium Framework:

hides all technical behind the scene and UI with the manual drag-drop, select

- While Selenium/Appium requires users to build common and reusable actions on their own, katalon Studio contains pre-defined sets of commonly used keywords or actions, users will find them enough to start implementing most test cases, but if they have more advanced or specific needs they still have a way to create custom keywords for their projects. Custom keywords are an extension of built-in keywords.

Why Choosing Katalon Studio Over QTP/UFT or Test Complete:

Katalon Studio scores fairly well as compared to commercial test automation tools such as UFT & Test Complete in multiple ways. It comes across as a viable, virtually free option to such tools available in the market:

Katalon Studio	UFT/Test Complete
No costs for licensing & maintenance	Cost-prohibitive solution: licensing & maintenance fees are quite high
Free upgrades	High fees for upgrades & add-on modules
Cross-platform solution	Only Windows based

Katalon Studio is a good choice for small and medium businesses. It's an evolving solution with many integrations that allow you to cover a variety of testing types with a single tool. It comes with all the necessary installations out of the box to run various types of tests, including API testing. It's free and easy to use. Katalon can be used by specialists with different QA engineering roles and varied programming skills, making it an attractive solution for teams with testers of different levels. Katalon Studio is mostly used to automate functional testing to reduce time spent on scripting. That said, because it has downsides like minor bugs and support for only one scripting language, it's not the best enterprise solution.

EXERCISE:

1. State the difference between Testcase & Test Suit
2. Why you preferred Katalon over any other Automation tool available in market ? state in your words also justify your answer with suitable example.
3. State the different views of Katalon GUI.
4. How many types of files you can attach to Katalon ? which feature of katalon allows you to attach file in katalon test suit.
5. what is the purpose of Object Spy and Object Repository ? Explain with suitable example

Lab 9: Create Test Case [Manual & Script View]

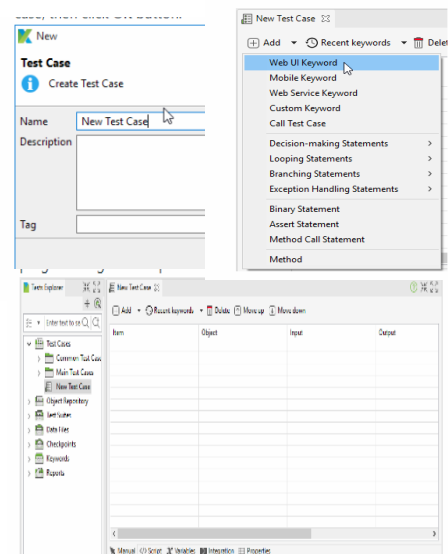
Katalon Studio supports Keywords-Driven testing where test cases consist of keywords that represent actions of users on the AUT (Applications Under Test). This allows users with less experience in programming to easily generate automation test. The below tutorial will give you step-by-step instruction in order to create an automation test case in manual mode.

Given a sample test case with the steps as below:

- *Open the browser*
- *Navigate to a website*
- *Click on certain control*
- *Validate if a control exists on the page*
- *Close the browser*

Follow these steps to automate the above test scenario in **Manual view**:

1. Select **File > New > Test Case** from the main menu. The **New Test Case** dialog will be displayed. Provide the name for the new test case, then click **OK** button.
2. Once a new test case is created, it is opened in **Manual view**. This view allows users to create automation tests easily with little programming skills required.
3. Select **Add > Web UI Keyword** from the command toolbar. Select the Open Browser keyword. This keyword will open a browser and navigate to the specified URL if provided. (selected keywords will have their description displayed along for reference)

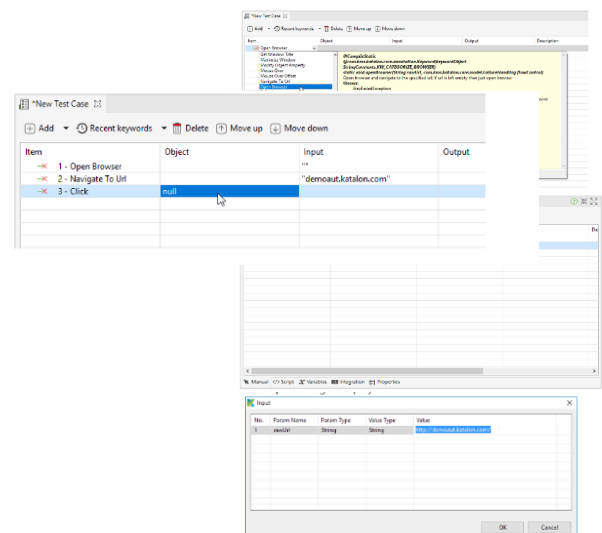


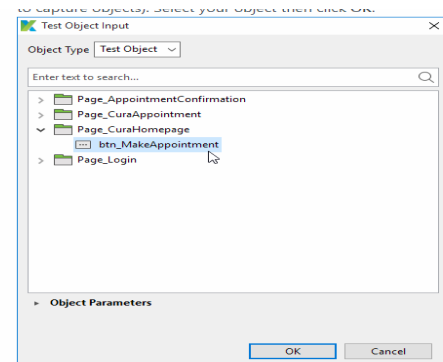
4. Add the Navigate To Url keyword. This keyword will navigate to a specified URL. Double click on the **Input** cell to provide additional data (parameters) for the keyword.
5. The **Input** dialog is displayed.

Where:

Field	Description
No.	The number of parameter for the selected keyword.
Param Name	The name of the parameter.
Param Type	The required data type for the parameter.
Value Type	The type of your input value (e.g.strings, variables, data sources...)

6. Add the **Click** keyword. This keyword represents the click action on a given object. Double click on the **Object** cell to provide the object for the keyword.
7. All captured objects in **Object Repository** are displayed in the **Test Object Input** dialog (Refer to **Spy Object** for details regarding how to capture objects). Select your object then





*New Test Case

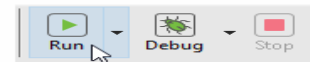
Add Recent keywords Delete Move up Move down

Item	Object	Input
1 - Open Browser		**
2 - Navigate To Url		"demoaut.katalon.com"
3 - Click	btn_MakeAppointment	
4 - Verify Element Present	btn_Login	0

*New Test Case

Add Recent keywords Delete Move up Move down

Item	Object	Input
1 - Open Browser		**
2 - Navigate To Url		"demoaut.katalon.com"
3 - Click	btn_MakeAppointment	
4 - Verify Element Present	btn_Login	0
5 - Close Browser		



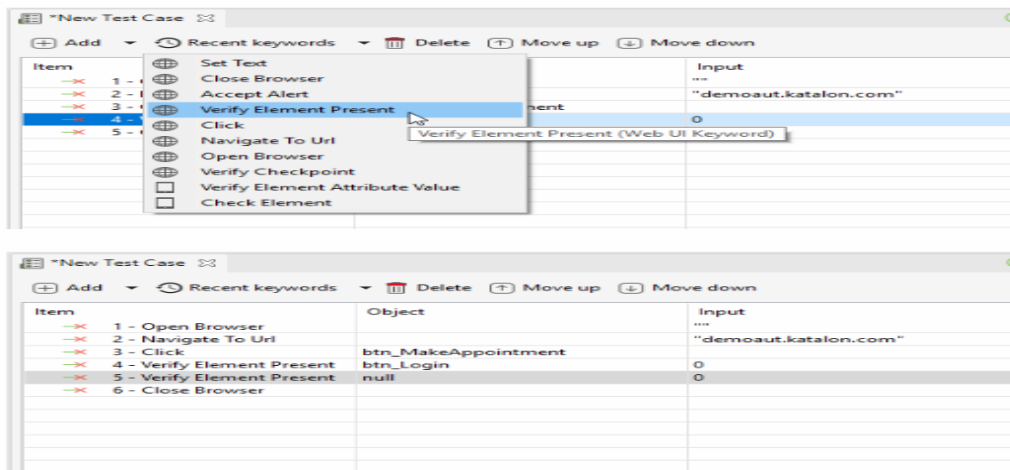
click **OK**.

8. Add the **Verify Element Present** keyword. This keyword validates if a certain object is displayed on the executing browser. Similar to the previous step, you need to specify the object to be used with this keyword.
9. Add the **Close Browser** keyword and save your test case.
10. Click on **Run** in the main toolbar to execute the test case.

Recent Keywords

Recent keywords allow users quickly add any of the last 10 recent used keywords in **Item** list. For example, let's take test case above. To add another *Verify Element Present* after *Step 4*, **recent keywords** feature would accomplish this in seconds.

Highlight *Step 4*. Click on **Recent Keywords** and select *Verify Element Present*. An extra step is added after *Step 4* as illustrated below:

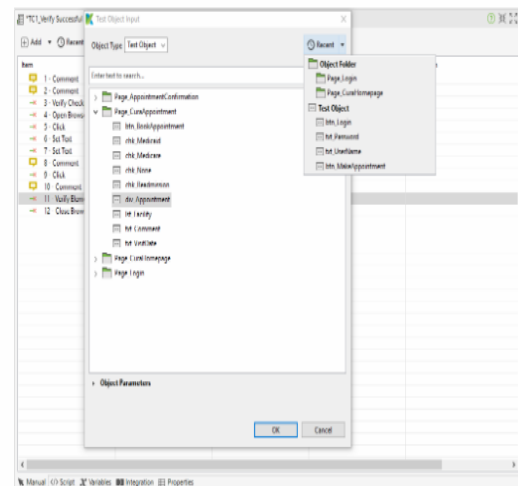


Recent Objects and Object Folders

Katalon Studio allows users to quickly select recently used **Object** or jump directly to recent used **Object Folder** in Object Repository.

Recent list will have two sections: **Object Folder** and **Test Object**

- **Test Object:** Display the names of the last 5 selected objects
- **Object Folder:** Display the names of 5 folders that contain any recent used objects



Script View:

Katalon Studio allows expert users to programmatically write automation test in the Script view of test cases. Users with Groovy/Java background can easily modify the test script as needed.

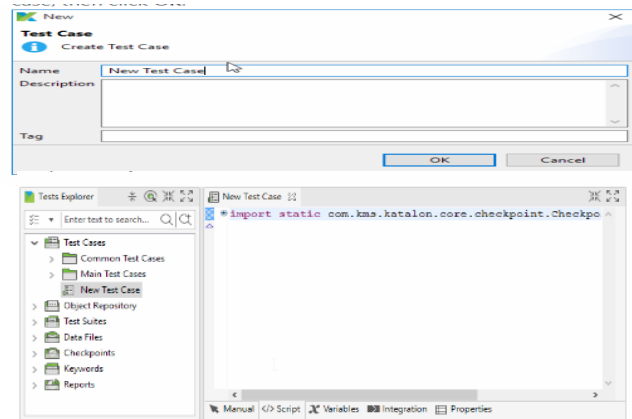
Given a sample test case with the steps as below:

- *Open the browser*

- *Navigate to a website*
- *Click on certain control*
- *Validate if a control exists on the page*
- *Close the browser*

Follow these steps to automate the above test scenario in **Script view**:

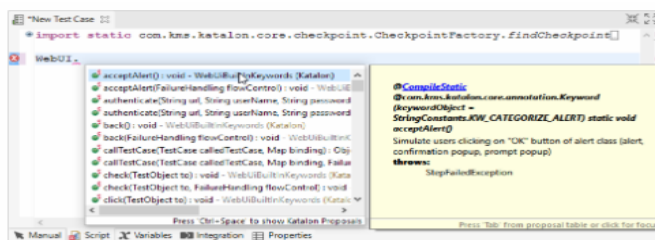
1. Select **File > New > Test Case** from the main menu. The **New Test Case** dialog will be displayed. Provide the name for the new test case, then click **OK**.
2. Once a new test case is created, you can switch to the **Script view** using the corresponding tab at the footer of the test case editor. Test steps specified in the **Manual view** are translated into a Groovy script in **Script view**.



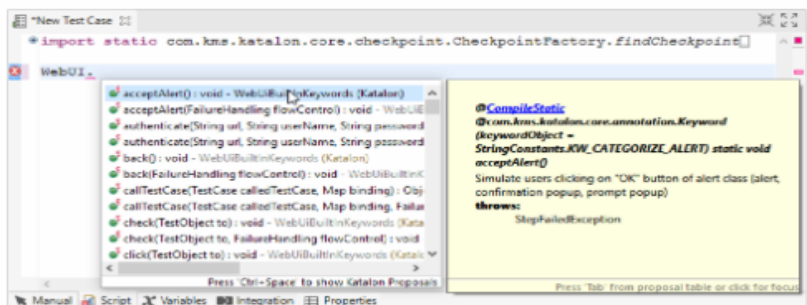
The import statement in a test script allows referencing to classes to be used. Expand the **import** section to see all default imported classes by Katalon Studio. The name after 'as' in each import statement is an alias for the class. You can change the alias for each class. These classes are necessary for composing a test script. Katalon Studio is an automation tool that supports keyword-driven testing. All keywords are grouped into WebUI, Mobile and WebService packages accordingly. Press **Ctrl + Space** to view these packages and functions from the imported classes.

3. In this scenario, you will create a Web application test script to make use of the Web UI built-in keywords. To use a built-in **WebUI**

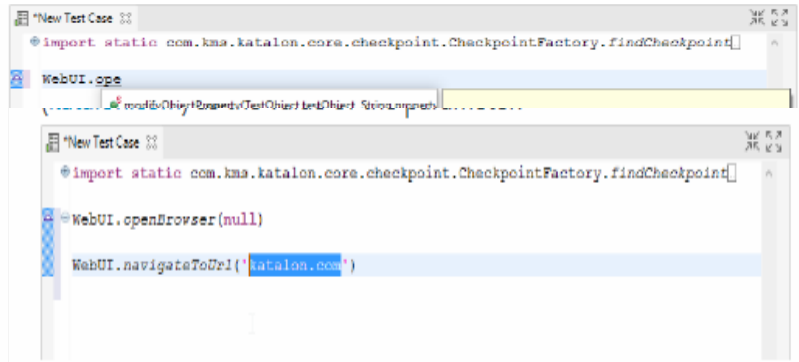
The **Content Assist** function will be invoked after users enter the dot character. **Content Assist** provides users with a context-sensitive suggestion for code completion. Therefore, all the built-in keywords for WebUI testing will be displayed as below:



4. After entering the dot character (.), testing will appear as shown in the



5. Select the **Open Browser** key specified URL if it is provided along for reference.
6. Enter the **Navigate To Url** keyword. This keyword navigates to a specified URL. For now, enter the URL of Katalon Studio (**katalon.com**) as the value of the parameter.

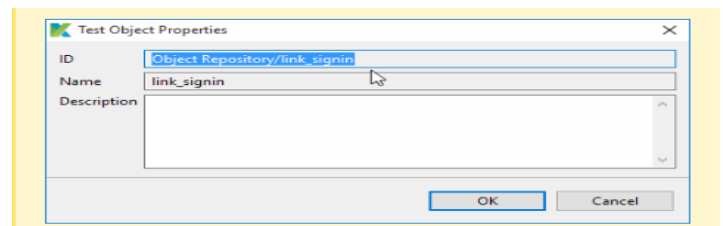


7. Enter the **Click** keyword. This keyword represents the click action on a given object. You need to specify an object for this action. Use the following syntax to refer to an object in Object Repository (alternatively, you can drag and drop the object to test case editor to generate the syntax):



findTestObject('{Object ID}')

Where Object ID is the ID of that object in Katalon Studio.

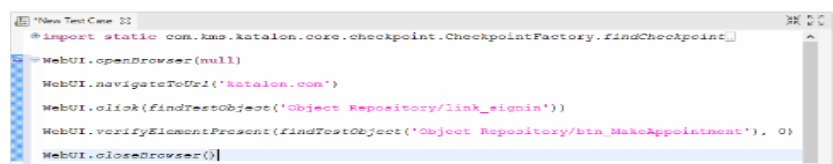


9. Enter the **Verify Element Present** keyword. This keyword validates if a certain object is displayed on the executing browser. Similar to the previous step, you need to specify the object to be used with this keyword.

keyword:



10. Add the **Close Browser** keyword and save your

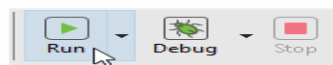


test case.

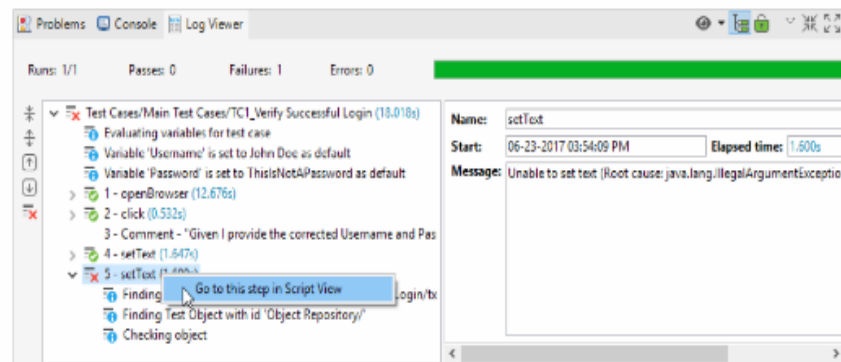
The following API docs may prove useful when working in Script view:

Class	Description
<u>Builtin Keywords</u>	List of common built-in keywords
<u>WebUI Builtin Keywords</u>	List of Web UI built-in keywords
<u>Web Service Builtin Keywords</u>	List of Web Service built-in keywords
<u>Mobile Builtin Keywords</u>	List of Mobile built-in keywords

- Click on **Run** in the main toolbar to execute the test case.



Katalon Studio should be able to execute all the steps of the sample test case. The test execution results are shown in **Log Viewer** as below:



EXERCISE:

1. Write Testcases for login/logout features (Atleast 6 combinations) of “Cura Health Care “ website . Use username= John Doe, Password=ThisIsNotAPassword ;

Also Print “Invalid credentials “for incorrect attempt on console.

/*attach Testreport and screenshot Here*

/***Hint:** use WebSpy to capture object and save it to object repository*/

Lab 10 : Variables (Local & Global) and Profiles

Test case variables

Test case variables are variables defined in a test case. In Katalon Studio, test case variables are defined in the **Variables** tab.

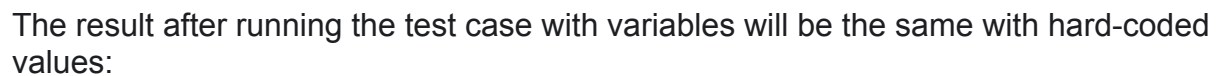
Instead of running test case with hard-coded values, you can create test case variables and dynamically run the test case with different inputs.

Manage test case variables

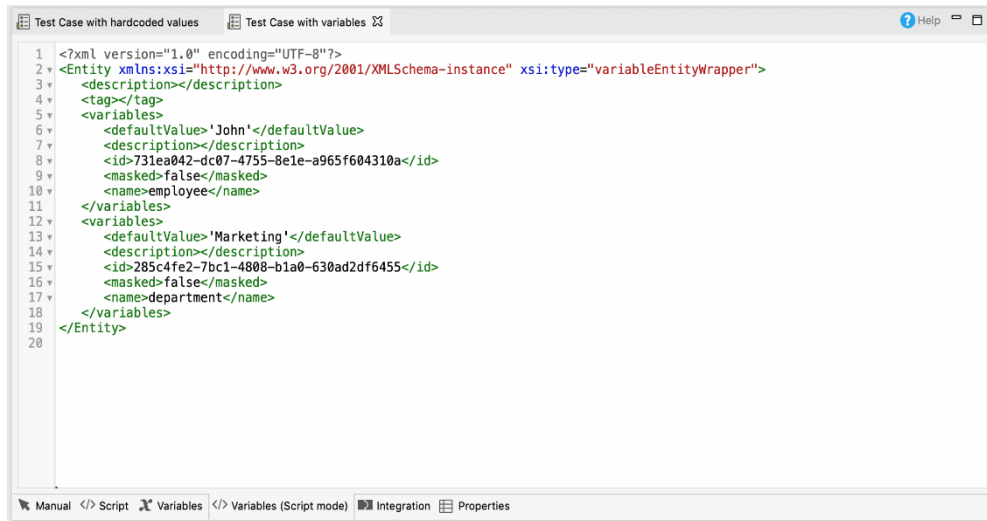
```
println "${employee} - ${department}"
```

To add variable using grid view, switch to **Variables** tab of your Test Case. Then click **Add**. A new row is added to the variable list.

Input variables and save the test case



Switch to **Variable (Script Mode)** tab, Katalon Studio will display a Script Editor with XML format. For example:

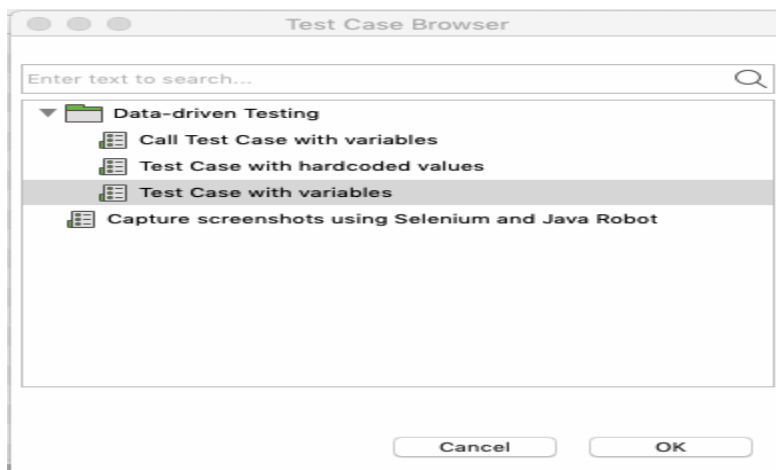


Call a test case with variables

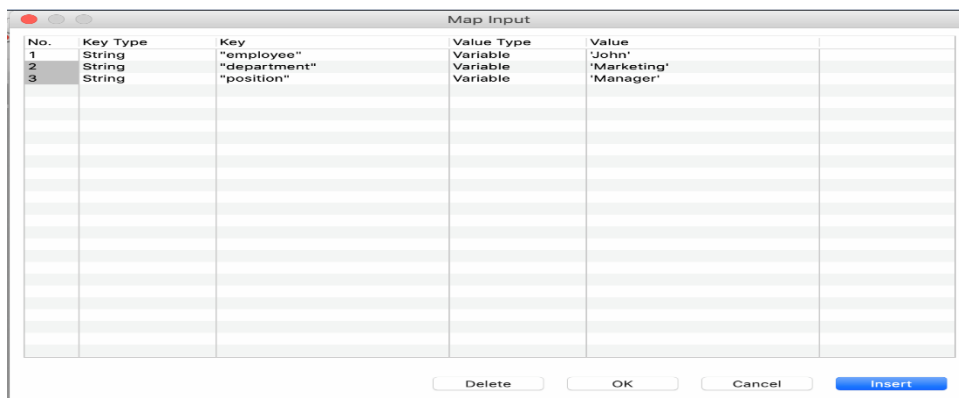
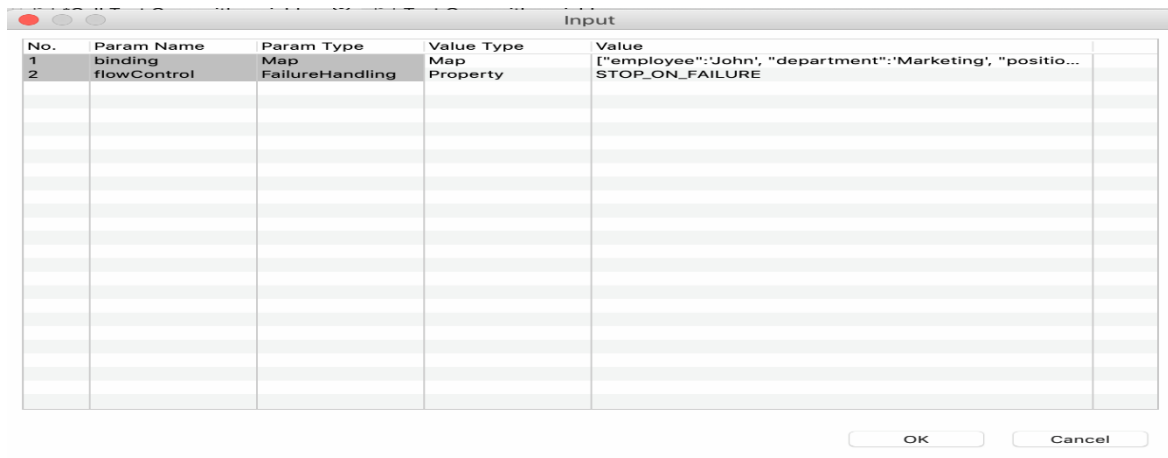
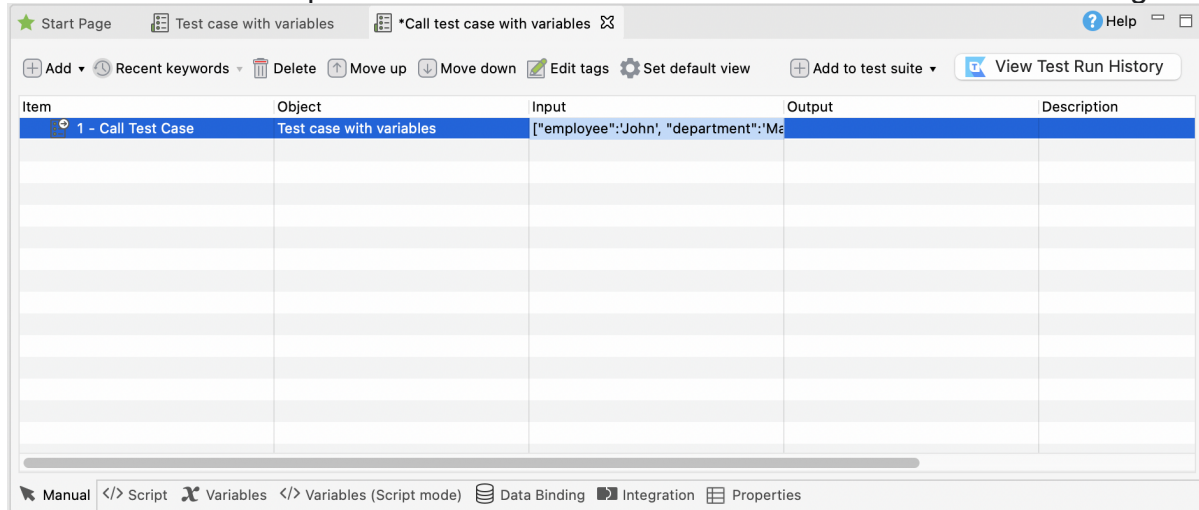
The following is an example of dynamically calling a test case with a set of values.

1. Open a test case in **Manual** view, then click **Add** and select option **Call Test Case**.
2. The **Test Case Browser** dialog which shows all existing test cases within the project will be displayed. Select the test case to be called and click **OK**.

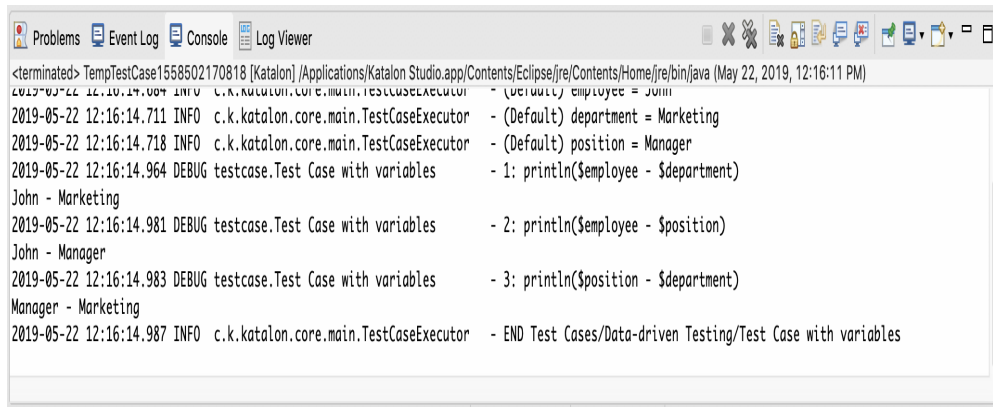
In the following example, we call the "Test Case with variables" test case.



A **Call Test Case** step will be added with the selected test case above as its target.



The result after running the test case will be displayed as below:



Global variables and Execution profile

Global variables are pre-defined values that are accessible across all test cases, test objects, web service objects, and email configurations in a project. They are used to execute test scripts in multiple and different environments.

For example, in a test case, you can use global variables to input data for keywords in Manual view (see highlighted in blue), or to input parameters when binding data for test execution (see highlighted in red).

```
import static com.kms.katalon.core.checkpoint.CheckpointFactory.findCheckpoint
CustomKeywords.'sample.Login.loginIntoApplicationWithGlobalVariable'()
WebUI.waitForElementPresent(findTestObject('Pages/Shop page/lnkShop'), GlobalVariable.waitPresentTimeout)
WebUI.click(findTestObject('Pages/Shop page/lnkShop'))

TestData product = findTestData(GlobalVariable.dataFile)
List<String> productList = product.getAllData().stream()
    .map{data -> data[0]}/*get first column of each row in data file */
    .collect(Collectors.toList())/*add collect and parse to list*/

for(def productName : productList){
    CustomKeywords.'sample.Shop.addToCart'(productName.toString(), GlobalVariable.urlProduct)
}
CustomKeywords.'sample.Checkout.CheckoutShopWithGlobalVariable'()
WebUI.closeBrowser()
```

In Katalon Studio, global variables are defined in execution profiles to configure the testing environment in terms of data and behaviors.

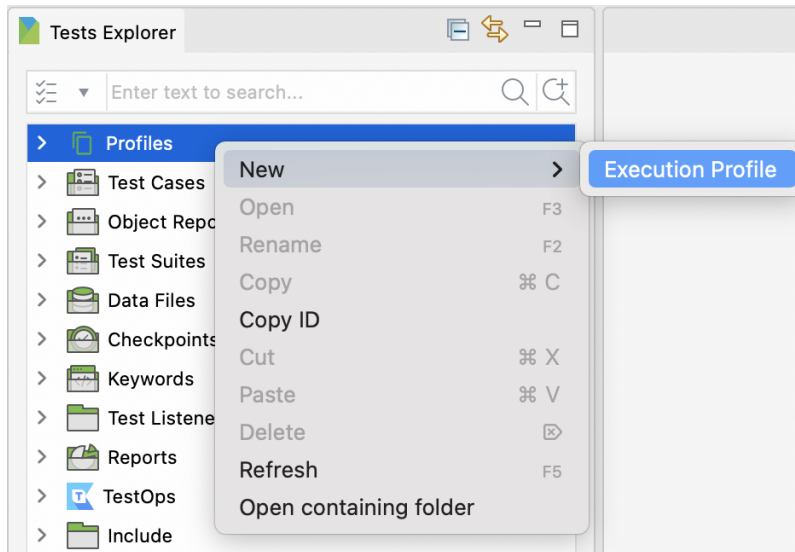
This document shows you how to work with global variables and execution profiles in Katalon Studio.

Create a profile

The following guide shows you how to create a profile to add global variables.

You can create execution profiles to manage global variables based on specific testing purposes..

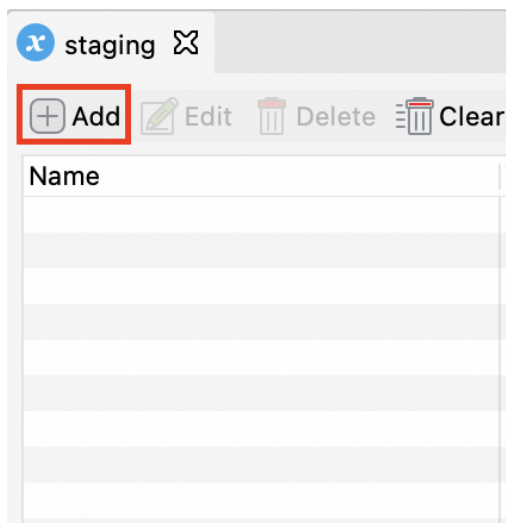
1. In the **Tests Explorer** tab, right-click on **Profiles > New > Execution Profile**.



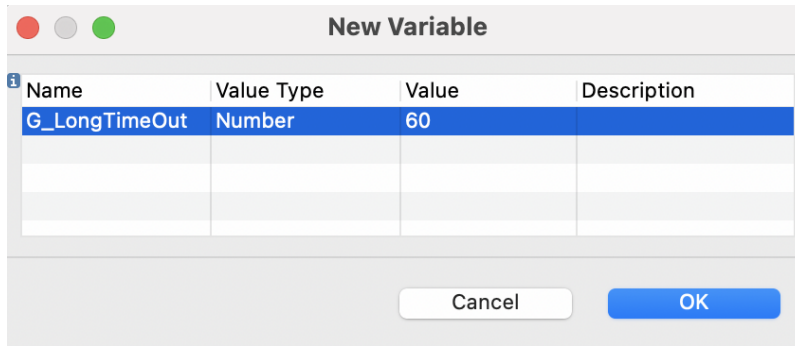
Enter a name for the profile in the **New Execution Profile** dialog.

2. In the profile manual view, click **Add**.

You need to define the profile content by adding global variables.



3. Specify details for the variable in the **New Variable** dialog, then click **OK**.

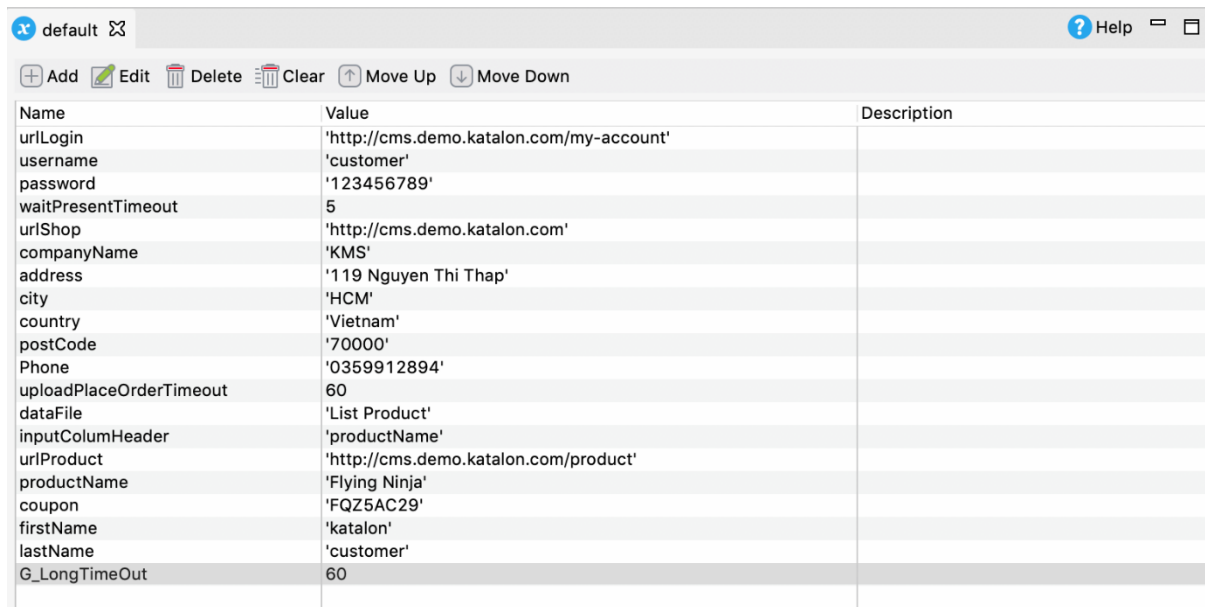


A dialog box titled "New Variable" with a table containing the following data:

Name	Value Type	Value	Description
G_LongTimeOut	Number	60	

At the bottom, there are "Cancel" and "OK" buttons.

- The variable is added to the profile accordingly.



A screenshot of an execution profile window titled "default". It contains a table with the following data:

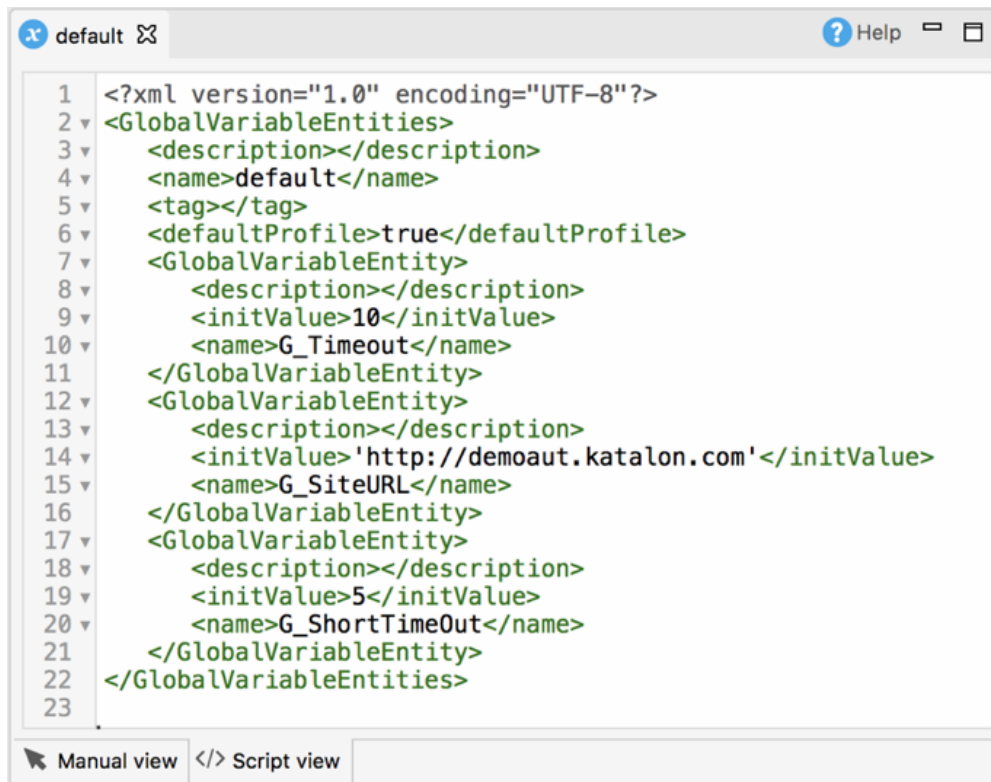
Name	Value	Description
urlLogin	'http://cms.demo.katalon.com/my-account'	
username	'customer'	
password	'123456789'	
waitPresentTimeout	5	
urlShop	'http://cms.demo.katalon.com'	
companyName	'KMS'	
address	'1 19 Nguyen Thi Thap'	
city	'HCM'	
country	'Vietnam'	
postCode	'70000'	
Phone	'0359912894'	
uploadPlaceOrderTimeout	60	
dataFile	'List Product'	
inputColumHeader	'productName'	
urlProduct	'http://cms.demo.katalon.com/product'	
productName	'Flying Ninja'	
coupon	'FQZ5AC29'	
firstName	'katalon'	
lastName	'customer'	
G_LongTimeOut	60	

You have created a new execution profile with your global variables.

View a profile

Execution profile is provided with **Manual view** and **Script view**.

In the **Script view**, an XML editor is available for adding variables via script. To ensure consistency in testing different environments, you can copy and paste the list of global variables from one profile to another.



```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <GlobalVariableEntities>
3   <description></description>
4   <name>default</name>
5   <tag></tag>
6   <defaultProfile>true</defaultProfile>
7   <GlobalVariableEntity>
8     <description></description>
9     <initValue>10</initValue>
10    <name>G_Timeout</name>
11  </GlobalVariableEntity>
12  <GlobalVariableEntity>
13    <description></description>
14    <initValue>'http://demoaut.katalon.com'</initValue>
15    <name>G_SiteURL</name>
16  </GlobalVariableEntity>
17  <GlobalVariableEntity>
18    <description></description>
19    <initValue>5</initValue>
20    <name>G_ShortTimeOut</name>
21  </GlobalVariableEntity>
22 </GlobalVariableEntities>
23
```

Manual view </> Script view

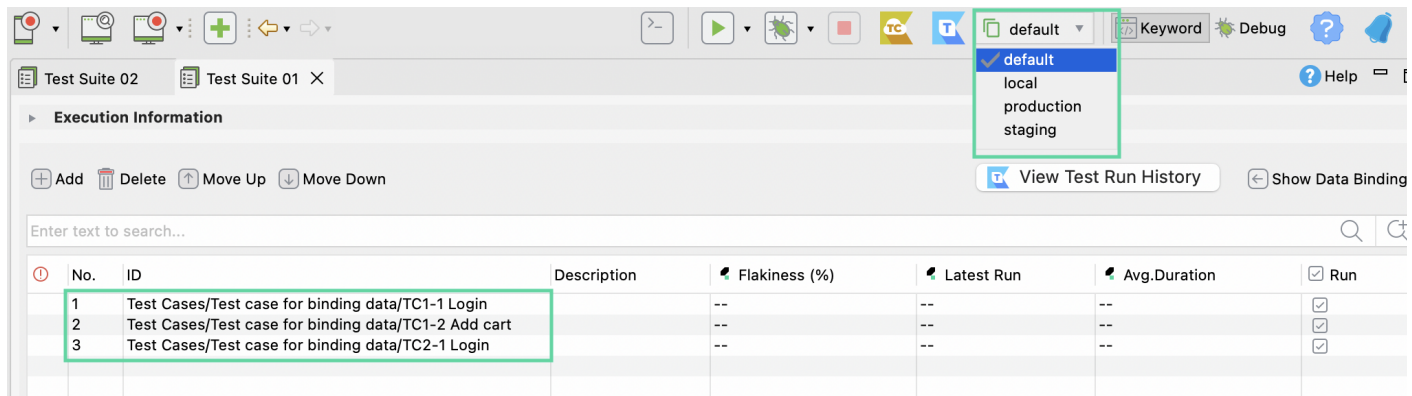
Use global variables in test execution

By selecting an execution profile, you can define global variables for executing a test case, test suite, and test suite collection in different scenarios. Katalon Studio uses the **default** profile by default as indicated on the main toolbar.

In test case and test suite

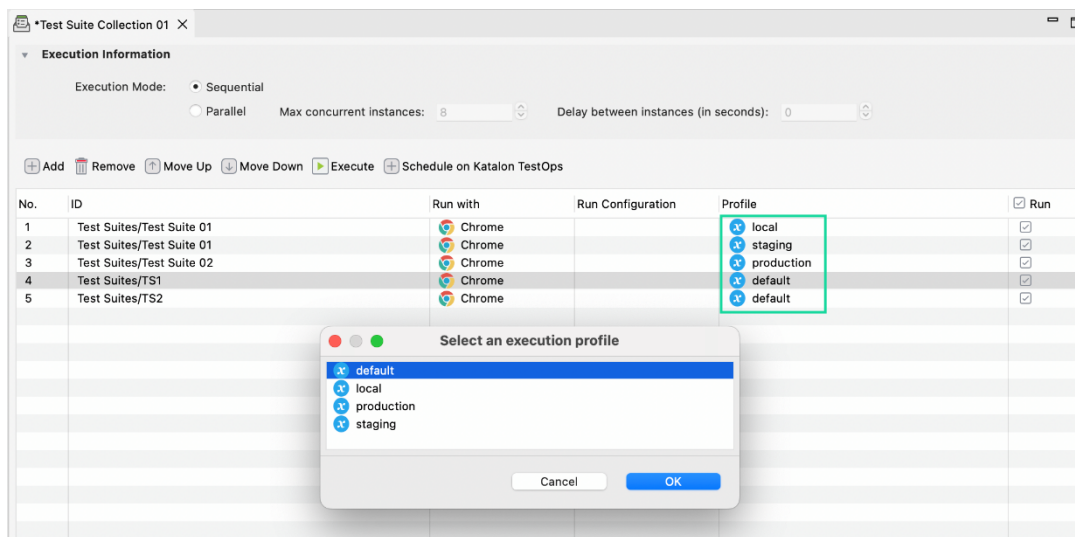
Select your desired profile from the main toolbar. This setting applies to all test cases and test suites within your project.

In a test suite, you can only select 1 specific profile to apply to all test cases in that test suite. For example, in this test suite, you can execute 3 test cases using the **default** profile at a time.



In test suite collection

In a test suite collection, you can assign which profile to apply to each test suite in the **Profile** column.



Scope of global variables

The scope of Global variables is applied in a test suite collection (TSC).

Katalon Studio defines global variables in an execution profile. In a test suite collection, you can assign different profiles to each test suite. During the runtime, modifying the value of global variables in one test suite will not impact the values of global variables in other test suites within the TSC.

You can find "Test Suites/New Test Suite (1)" is listed twice in the following screenshot. The first one uses "default," and the second one has "staging". Any changes made in profile "default" will not impact profile "staging". These two test suites run

independently without affecting each other. This association confirms **Global variables and Profile are Test Suite scoped**.

sample *New Test Suite Collection

Execution Information

Execution Mode: ☒ Sequential ☐ Parallel Max concurrent instances:

Add Remove Move Up Move Down Execute Schedule on Katalon TestOps

No.	ID	Run with	Run Configuration	Profile
1	Test Suites/New Test Suite (1)	Chrome		default
2	Test Suites/New Test Suite (1)	Chrome		staging
3	Test Suites/New Test Suite	Chrome		production
4	Test Suites/STD-375/ErrorTestSuite1	Chrome		staging

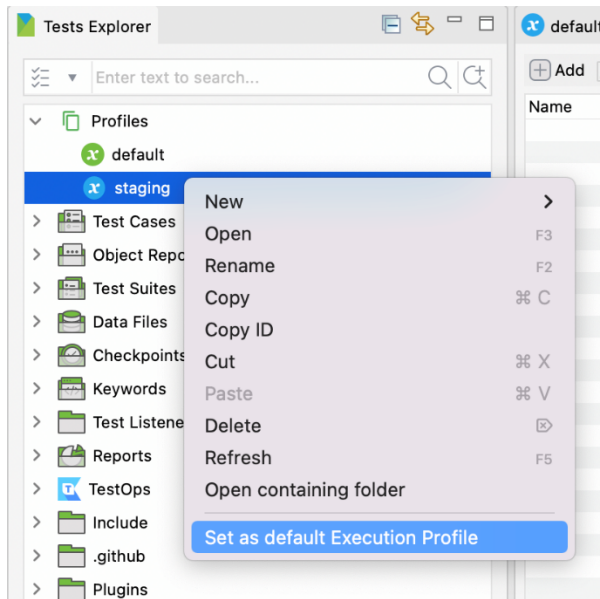
Set default profile at project level

From Katalon Studio v7.4.2, you can set a default profile at the project level.

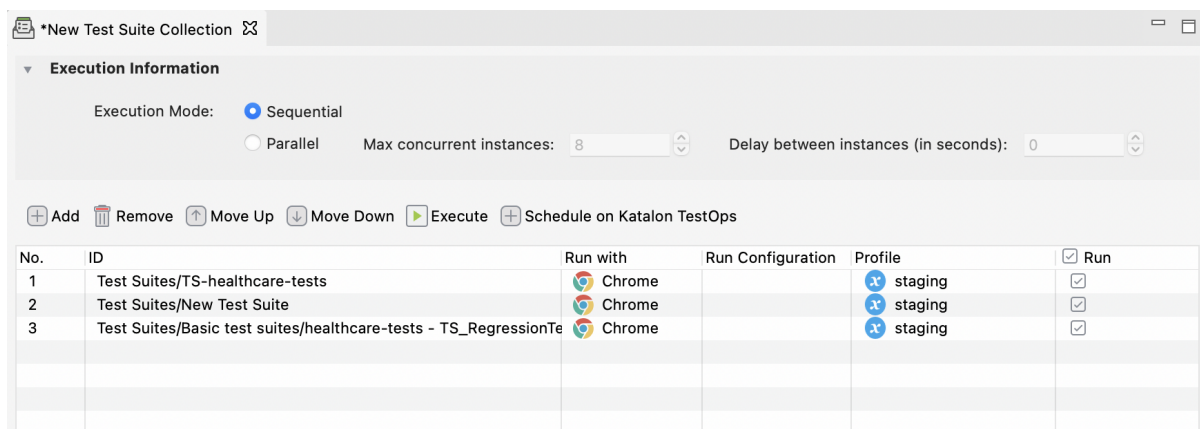
A default profile is where you store commonly used global variables. Other profiles can either inherit or override the global variables stored in the default one.

You may have multiple profiles for executing your tests, for instance, *staging* and *production* profiles. It would be convenient to set a profile as your default one in every execution of a project.

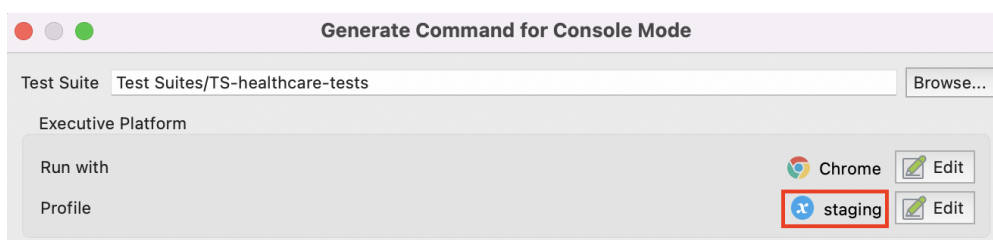
Right-click on your desired execution profile and select **Set as default Execution Profile**.



The selected profile becomes a default execution profile for test case, test suite, and test suite collection in that project.



The setting is also applied to Command Builder for Katalon Runtime Engine.



Exercise:

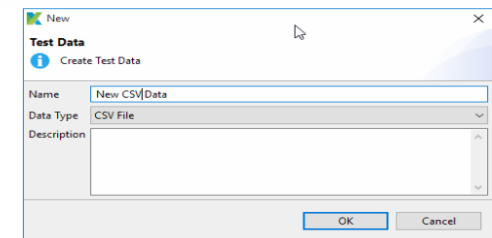
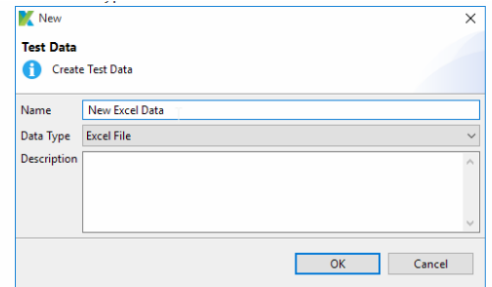
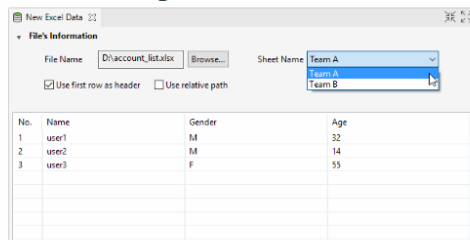
1. Set up execution profile for QA environments and run tasks(below) under QA profile.

2. Define global variables for base URL, login credentials, use it in a testcase and execute.
3. Run the same test case(under different profiles and compare responses
4. Create local variables for appointment time slots during test case execution.
5. Simulate failed login attempts with incorrect credentials — catch authentication exceptions.
6. Attempt booking with invalid data (e.g., empty fields, past date) — validate input exceptions.
7. Disconnect network mid-session — test timeout and connection exceptions.

Lab 11: Data Driven Testing

Create an Excel Test Data

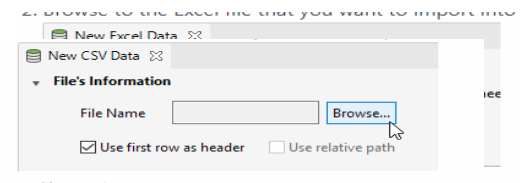
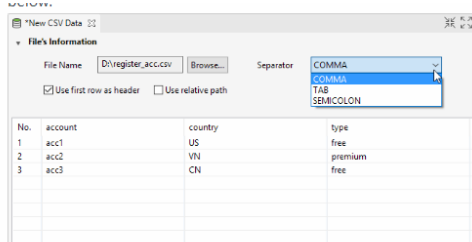
1. Select **File > New > Test Data** from the main menu. The **New Test Data** dialog is displayed. Enter the name for your test data and select **Data Type** as **Excel File**. Click **OK**.
2. Browse to the Excel file that you want to import into Katalon Studio. Data from the selected Excel file is populated into the **preview section** below.



3. Save the Test Data when you finish. The data set defined here can be utilized in other configurations. For example, you can use it to input data for keywords in Manual View or data for Test Execution when setting up test suites.

Create a CSV Test Data

1. Select **File > New > Test Data** from the main menu. The **New Test Data** dialog is displayed. Enter the name for your test data and select **Data Type** as **CSV File**. Click **OK**.
2. Browse to the CSV file that you want to import into Katalon Studio. Data from the selected CSV file is populated to the **preview section** below.

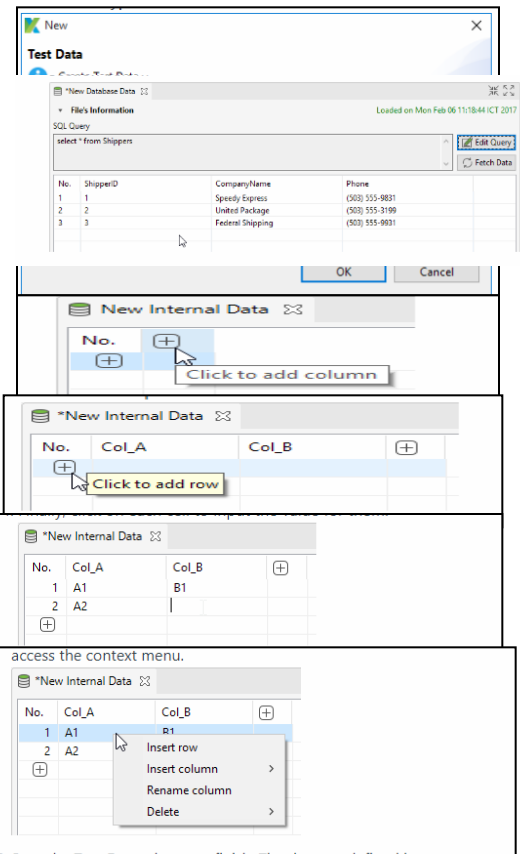


3. Save the Test Data when you finish. The data set defined here can be utilized in other configurations. For example, you can use it to input data for keywords in Manual View or data for Test Execution when setting up test suites.

Create an Internal Test Data

With **Internal Data**, you can freely define the data in tabular format. It's up to you to decide how many columns, rows, and what value for each cell.

1. Select **File > New > Test Data** from the main menu. The **New Test Data** dialog is displayed. Enter the name for your test data and select **Data Type** as **Internal Data**. Click **OK**.
2. In the Editor View, select the option to add a new column.
3. Select the option to add a new row.
4. Finally, click on each cell to input the value for them.
5. You can edit or delete the columns (or rows) by right-clicking to access the context menu.
6. Save the Test Data when you finish. The data set defined here can be utilized in other configurations. For example, you can use it to input data for keywords in Manual View or data for Test Execution when setting up test suites.

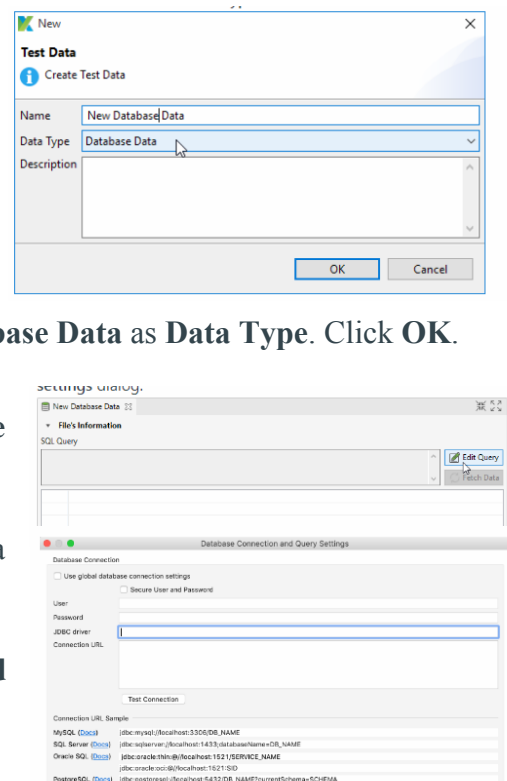


Create a Database Data

With **Internal Data**, you can freely define the data in tabular format. It's up to you to decide how many columns, rows, and what value for each cell.

1. Select **File > New > Test Data** from the main menu. The **New Test Data** dialog is displayed. Enter the name for your test data and select **Database Data** as **Data Type**. Click **OK**.
2. Click **Edit Query** to open the **Database Connection and Query settings** dialog.
3. Enter the connection details as well as the data query then click **OK**.

Starting from **Katalon Studio version 7.0.0** and



later, you can query data from additional database sources with the **JDBC Driver** field in the dialog. You can enter the ClassDriverName of the database that has a library support connection (JDBC).

4. Queried data is fetched and loaded respectively into the **preview section**.

5. Save the Test Data when you finish. The data set defined here can be utilized in other configurations. For example, you can use it to input data for keywords in Manual View or data for Test Execution when setting up test suites.

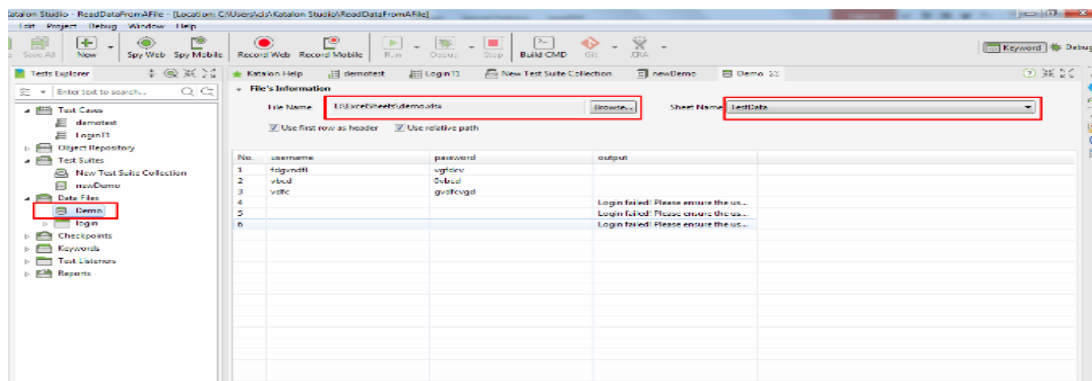
How to use Data from Excel File in Katalon:

Step 1: create a new data file(excel) example “demo.xlsx”.

Step 2: And enter some data into that file.

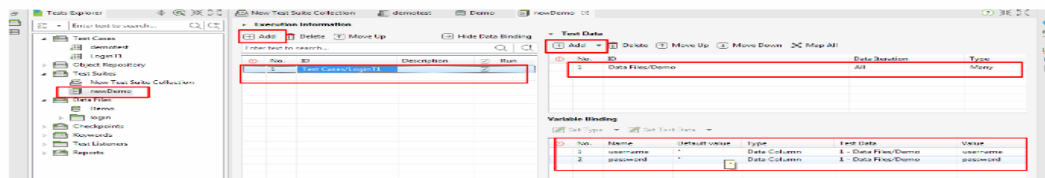
Step 3: Create a new data file in katalon studio.

Step 4: And browse your external file and select the sheet.



Step 5: Create test case and **define some variable in the test case**.

Step 6: Now create a TestSuite and configure a test case. And bind the variables with the column name.



Step 7. Set up a project. You can choose to create your new project either from scratch or from one of project samples (available with katalon studio).

Go to File > New > Project, then select your preference. Insert the project's name, the location to store its data and description (optional).

Step 8. Create your first test case To create a test case, right-click Test Cases > New > Test Case To simplify and speed up the process of writing test cases, especially in high volume, you can

use Katalon Studio standard format. For example:

```
WebUI.openBrowser('https://katalon-demo-cura.herokuapp.com/')
```

```
WebUI.click(findTestObject('Page_CURA Healthcare Service/a_MakeAppointment'))
```

```
WebUI.setText(findTestObject('Page_CURAHealthcareService/input_Username_username'), 'John Doe')
```

```
WebUI.setEncryptedText(findTestObject('Page_CURA Healthcare Service/input_Password_password'), 'g3/DOGG74jC3Flrr3yH+3D/yKbOqqUNM')
```

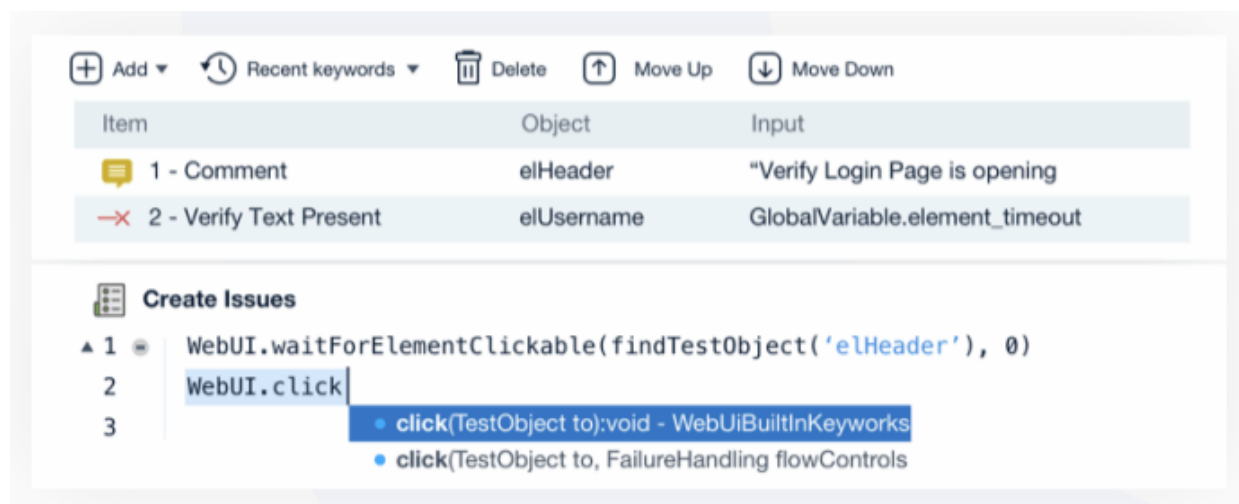
```
WebUI.click(findTestObject('Page_CURA Healthcare Service/button_Login'))
```

Step9. Click Run to see how it works.

Next, we will go through the “ChromeDriver,” “click” and “findTestObject” methods, and how to use the “Page_CURA Healthcare Service/a_Make Appointment” argument to begin testing with Katalon Studio.

- Browser drivers: Browser drivers, i.e., ChromeDriver are abstracted away from the code and set up at the execution time so that the code is enabled for cross-browser testing.
- Click and setText: This is an example of Katalon Studio’s **open-source built-in keywords** library to wrap and enhance the limited set of Selenium keywords. You can also create your own set of custom keywords or import the shared custom keywords made by other users.
- Page_CURA Healthcare Service/ a_Make Appointment: are the test objects designed to follow the Page Object Model pattern. Katalon Studio supports your test objects with the “auto-healing” feature to make them sustainable, despite the continuously changing nature of the AUT. The script and objects above can be quickly generated by the Recording and Spying

Katalon Studio provides a dual-script interface for both manual and automated tests:



In practice, a team of testers and developers can apply these steps to their complete workflow:

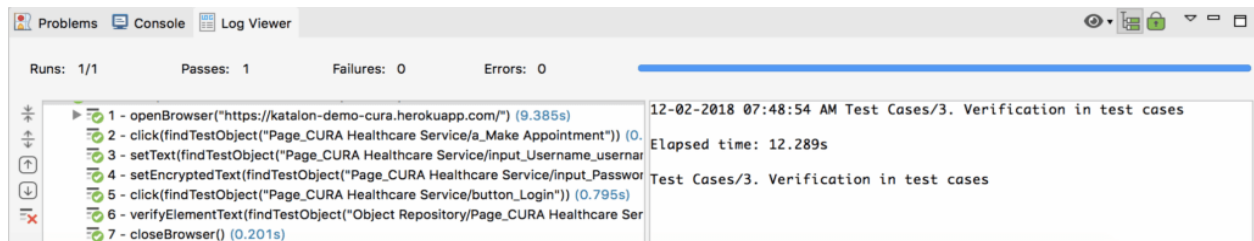
- Firstly, the automation experts will prepare all the scripts needed to build test cases such as the custom keywords, test listeners, calling test cases, etc.
- Then, these prepared scripts will be transferred to the manual QAs to utilize without having to handle scripting.
- As a result, the manual QAs can gradually learn how to script and become automation scripters.

Verification in the test case: To get the result of the login process, you need to add the verification script to the test case. As the fundamental of scripting in Katalon Studio for the first test case has been explained, we can move on to another test case: “Make Appointment” header verification. You can leverage Katalon Studio’s rich set of verification keywords to work on that requirement. Note that this kind of script can be done in both Manual and Script mode.

Item	Object	Input
1 - Open Browser		"https://katalon-demo-cura.herokuapp.com/"
2 - Click	a_Make Appointment	
3 - Set Text	input_Username_username	"John Doe"
4 - Set Encrypted Text	input_Password_password	"g3/DOGG74jC3Flrr3yH+3D/yKbOqq"
5 - Click	button_Login	
6 - Verify Element Text	h2_Make Appointment	"Make Appointment"
7 - Close Browser		

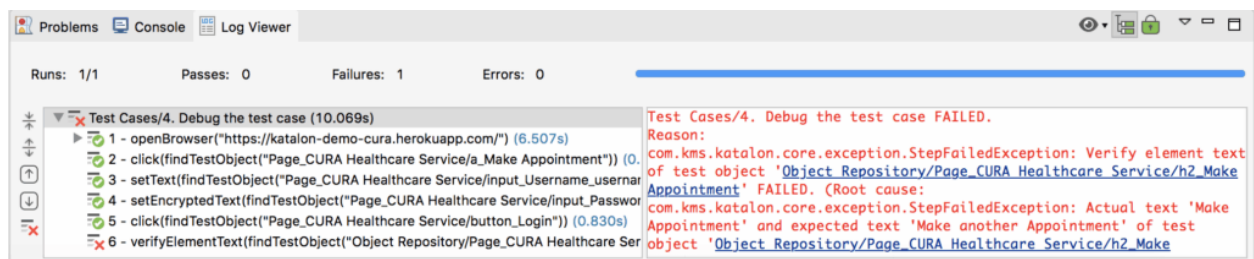
WebUI.verifyElementText(findTestObject('Object Repository/Page_CURA Healthcare Service/h2_Make Appointment'), 'Make Appointment', FailureHandling.STOP_ON_FAILURE)

You can then check the execution results in the Log Viewer



Debugging the test case: Try changing the verification text to “Make another Appointment.” to make the test case fail. Here are a few options of how you can check the reasons for failure in Katalon Studio:

- Option 1: Investigate error logs



- Option 2: Debug mode

For complex cases, Katalon Studio offers a debug mechanism that works similarly to the code debug mechanism in the advanced developer IDE.

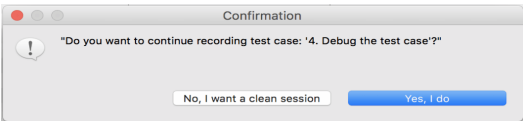
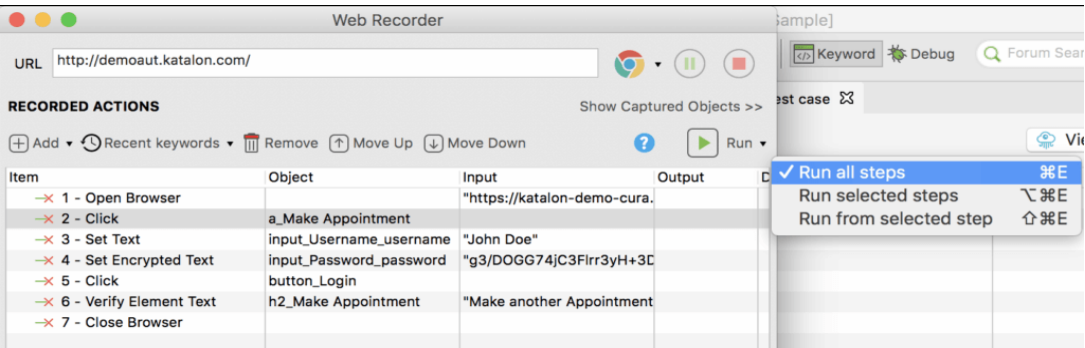


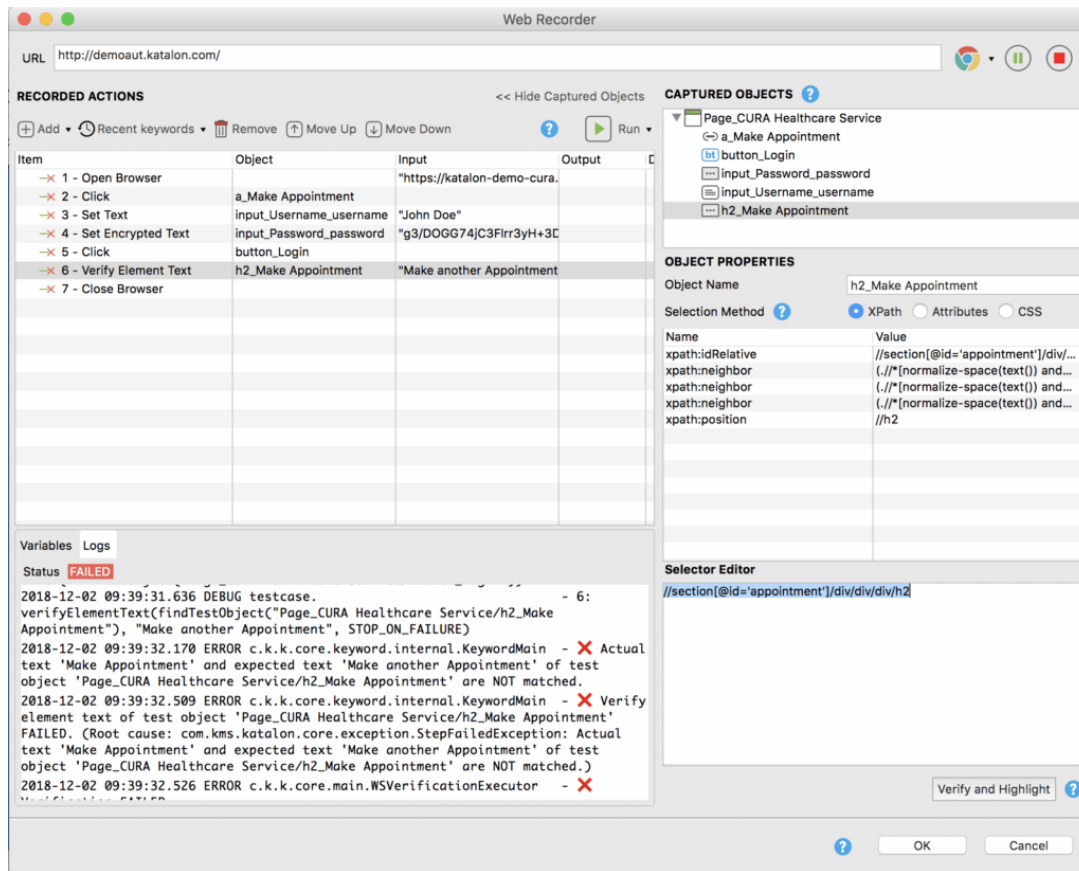
- Option 3: Manual debug

It is possible to use the ‘Recording’ feature of Katalon Studio for debugging. The tool supports recording the failed test case, running the error test step for you to analyze and fix the issue directly in the recording mode.

When your test case failed, click “Yes, I do” in the window that shows up to continue recording the failed test cases.

Choose Run all steps to run the error test step. After that, you can investigate and fix the issue directly in the recording mode.





Plan the test case in the test suite: Right-click Test Suites > New > Test Suite

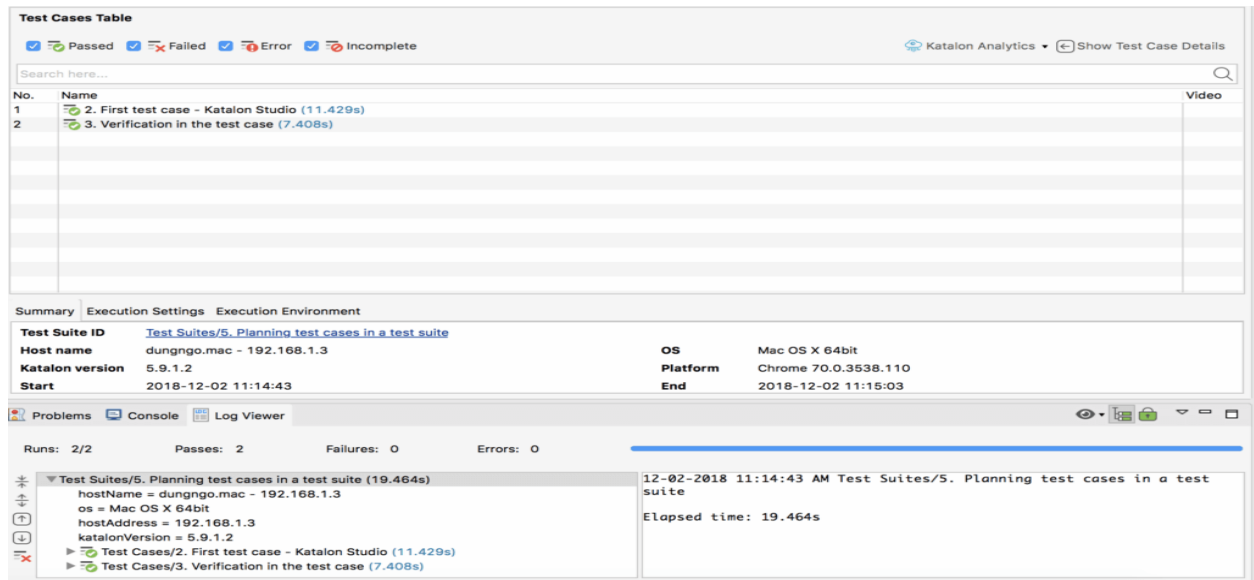
To create test cases with multiple configurations (e.g., retrying on failure, email sending or data-driven binding), use the test suite capability. The added configurations of a test suite execution can be managed by expanding the Execution Information section. See the example below:



Execute the test suite and view the result: Select Test Suite > Run

The final step is to execute a test case in the designed test suite:

- After being successfully planned, the test suites or test suite collections can be executed not only directly in Katalon Studio, but also by the command line.
- Combining the standard JUnit format of the execution results and pre-built Docker images, these features will give you the flexibility to integrate Katalon Studio into a CI/CD pipeline with tools such as Jenkins or CircleCI.



Exercise:

1. Attach an excel sheet "Testdata" (having Username and password) , Create and execute testcases for cure health ;generate and attach Testreport/*Attach screenshot/printout here*/ Testdata must have two invalid and one valid password, Use any loop through scripting language.

2, use excelSheet(1 a) and write “valid” [if the username & password is correct] or” invalid” [if the username & password is incorrect] from katalon.

3..Create internal file having data values for user names and password ; Execute TCs to read test data values from internal file.

Lab 13 : API Testing

API testing (or Web service testing in the context of a Web application) has become more important in software testing. The interest in API testing has been increasing over the last five years. This trend possibly indicates that the demand for applying API testing has become more prevalent. Testing API or web services is no longer performed solely by the original developer. This activity is now a common practice among outsourcing teams who independently verify and validate their products.

Components in an API test project

Before creating our first API test, let's have a look at the format we use to set up a testing project.

Object repository

- Object repository is a place which stores all the Web service endpoints along with all information of request method, URL, header, content, and authentication.
- Web service test objects in object repository are integrated by a folder system for better management.

Test cases

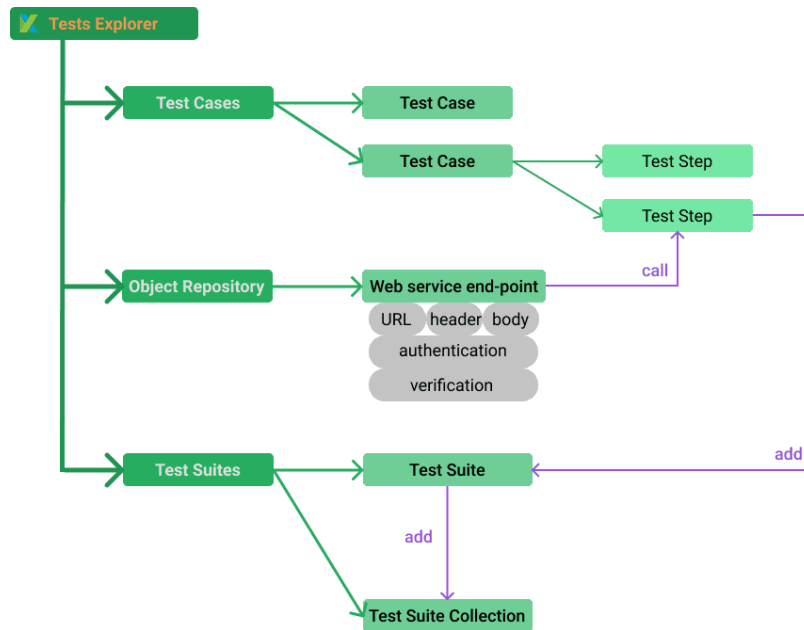
- Test cases stores all test scenarios and is grouped by a folder system. Each test case includes a few steps illustrating a test scenario.
- We can execute a test case individually with a specified execution profile.

Test suites

- Test suites is the place where all test suites are stored. A test suite is a collection of test cases verifying a specific target.
- Test cases at the test suite level can be executed with a data-driven approach.
- Test reports are also generated at the test suite level

Test suite collection

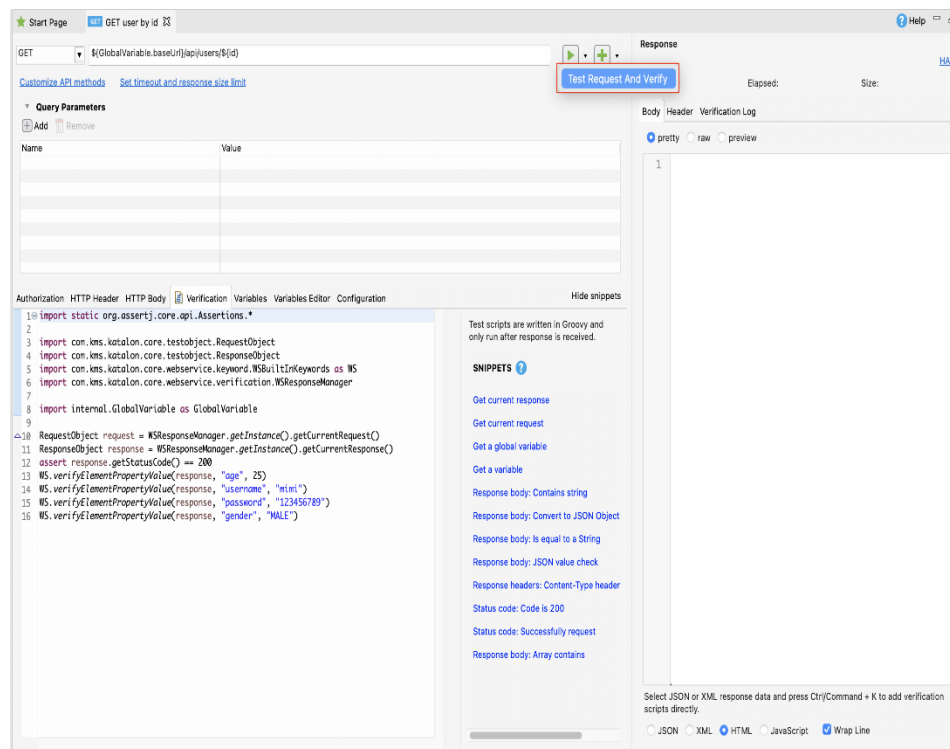
- Test suite collection is a collection of test suites verifying a larger target.
- Test suite at test suite collection level has specific test environments specified.



RESTful endpoint in object repository

Katalon Studio stores Web service endpoint for testing at Object Repository, which is similar to Test Object in UI Test.

There are some important concepts needed to specify when testing a RESTful request:



Request method

We can choose one of these following methods for your first request test: GET, POST, PUT, DELETE. The method needs to match with the URL to have a valid request. For instance, let's assume that our first test is a public API from Jira Cloud version. We should use the GET method to receive information on an existing ticket using its ID.

Request URL

Along with the request method, request URL is used to tell the web server which API is utilized under test. Any mismatch between method and URL will lead to invalid request exception at runtime or wrong data response.

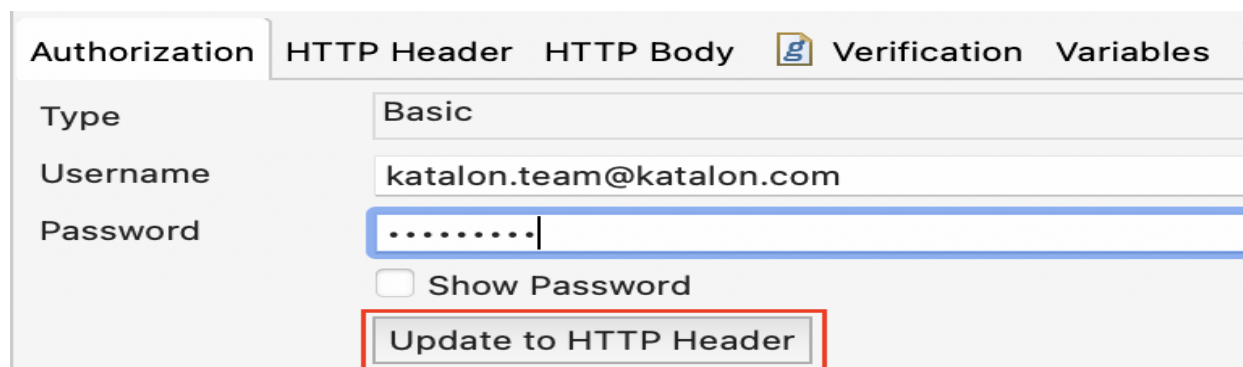
Follow redirects

The Follow redirects option allows you to control how your requests handle HTTP 3xx status code responses. This option ensures accurate test results and manages the flow of requests. When enabled, Katalon Studio automatically follows the redirection path provided by the server, ensuring that your test reaches the final destination without additional intervention. When disabled, the request stops at the redirection response, allowing you to manually handle and inspect the redirection URL.

Authorization

Authorization is an essential part of an API. It is used to get the correct data under permission (unless the data is public). Katalon Studio supports common authentication methods:

The basic method requires username and password. Don't forget to click Update to HTTP Header so that the authentication can be applied to HTTP Header.



The screenshot shows the 'Authorization' tab in Katalon Studio. The 'Type' is set to 'Basic'. The 'Username' field contains 'katalon.team@katalon.com'. The 'Password' field is masked with dots. Below the password field is a checkbox labeled 'Show Password' which is unchecked. At the bottom, there is a button labeled 'Update to HTTP Header' which is highlighted with a red rectangular border.

Authorization	HTTP Header	HTTP Body	Verification	Variables
Type	Basic			
Username	katalon.team@katalon.com			
Password				
☐ Show Password				
Update to HTTP Header				

Authorization	HTTP Header	HTTP Body	Verification	Variables	Variables Editor	Configuration
+ Add Remove						
Name		Value				
Authorization		Basic YfsndflsdfdknfsdSJJFFMFKLdfknfdkjdfsd				

Verification

Verification is the place where you define the assertion to ensure the response will contain the expected information.

The screenshot shows the Katalon Studio interface for a test case. The top navigation bar includes tabs for Authorization, HTTP Header, HTTP Body, Verification (selected), Variables, Variables Editor, and Configuration. Below the tabs is a table for adding verification items. The main area is divided into three sections:

- Script Editor:** Contains Groovy code for assertions. The code includes imports for `org.assertj.core.api.Assertions`, `com.kms.katalon.core.testobject.RequestObject`, `com.kms.katalon.core.testobject.ResponseObject`, `com.kms.katalon.core.webservice.keyword.WSBuiltInKeywords`, and `com.kms.katalon.core.webservice.verification.WSResponseManager`. The script then uses `WSResponseManager` to get the current request and response, and `assert` to verify the status code and response body properties (age, username, password, gender).
- SNIPPETS:** A list of quick assertions such as "Get current response", "Get current request", "Get a global variable", "Response body: Contains string", "Response body: Convert to JSON Object", "Response body: Is equal to a String", "Response body: JSON value check", "Response headers: Content-Type header", "Status code: Code is 200", "Status code: Successfully request", and "Response body: Array contains".
- Response and Verification Log:** The right panel shows the response body (pretty, raw, or preview) and the verification log.

Verify the RESTful object

The verification tab of a request is similar to the Script tab of a test case. In other words, you can write custom scripts with built-in keywords or Groovy/Java scripts to verify the response data. Besides built-in keywords, Katalon Studio also supports built-in snippets, which help you to generate assertions with a single click. It is useful for testers who might find it difficult to deal with parsing and to assert with JSON data format.

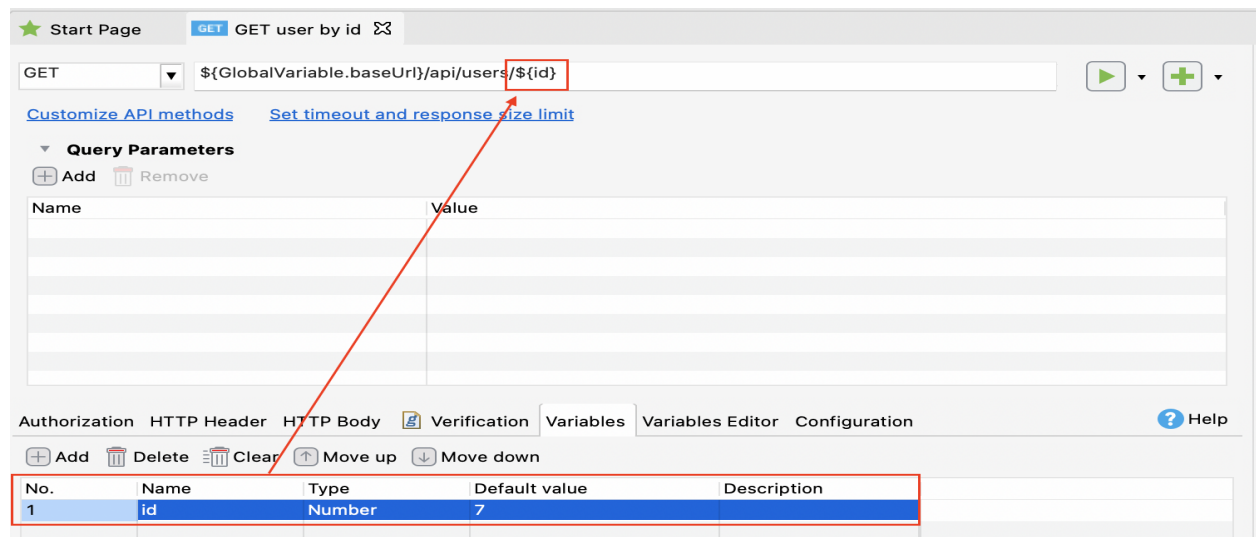
The right panel of the request is the response displayed in nice format automatically and the result of verification at Verification Log. To include verification script when sending the request, you need to choose the Test Request and Verify option from the execution button.

The Verification script helps you have quick feedback of the request status rather than an actual test. You can add more assertions to the test case level in the next step.

Variables

Variables make API testing more robust and dynamic with the data-driven approach. In Katalon Studio, every part of the request can be parameterized. In other words, dynamic data can be used for: URL, Authentication, HTTP Header, and HTTP Body to maximize the capability of data-driven testing. Following setup works the same with the above example:

Variables in the RESTful object



The screenshot shows the Katalon Studio interface for a RESTful API test. The URL is set to `${GlobalVariable.baseUrl}/api/users/${id}`. A red box highlights the `${id}` variable in the URL, and a red arrow points from it to the 'Variables' tab. The 'Variables' tab shows a table with one variable: 'id' of type 'Number' with a default value of '7'.

No.	Name	Type	Default value	Description
1	id	Number	7	

Formatter

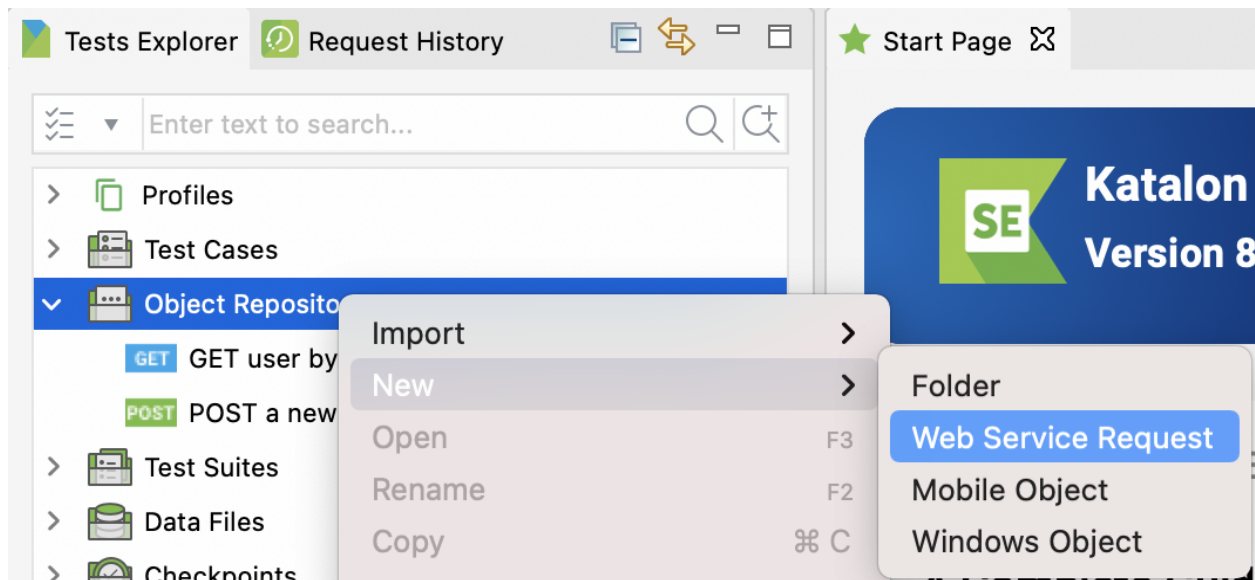
The response will be automatically displayed in a neat format: JSON, XML, HTML, and JavaScript. It is helpful for a quick view of the response status.

Create API tests in Katalon Studio

This section shows you how to create API tests in Katalon Studio.

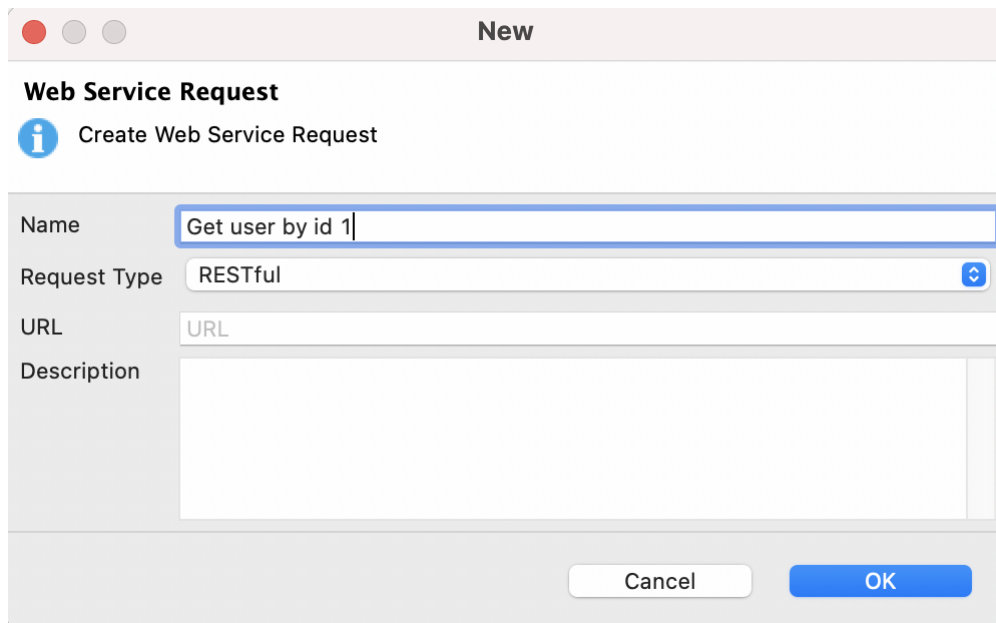
To create API tests in Katalon Studio, do as follows:

1. Open a Katalon project in Katalon Studio.
2. Create a new RESTful endpoint at Object Repository: go to **Object Repository > New > Web Service Request**.



1. At the **Create New Web Service Request** dialog, you can either choose to create a RESTful or a SOAP request.

Request type is a required field. You need to specify it exactly at this step. In contrast, **URL** is not required. You can set this value later in the next step.



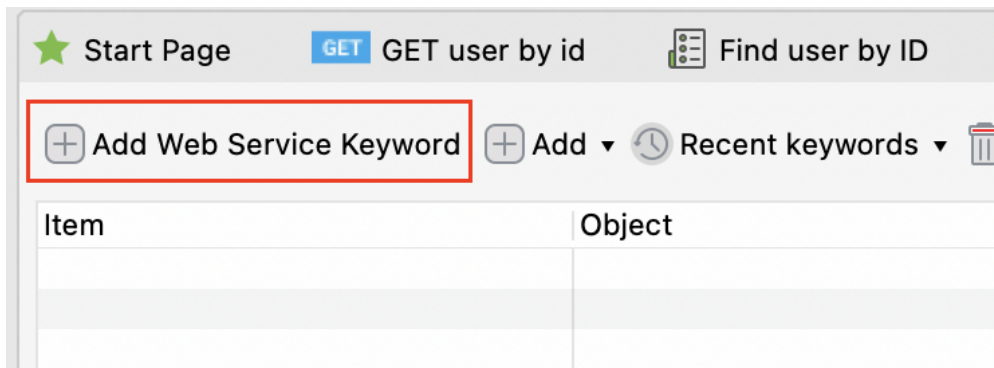
Click **OK**, then we are ready to input more details to the first RESTful test.

2. Create a new test case with an existing request.

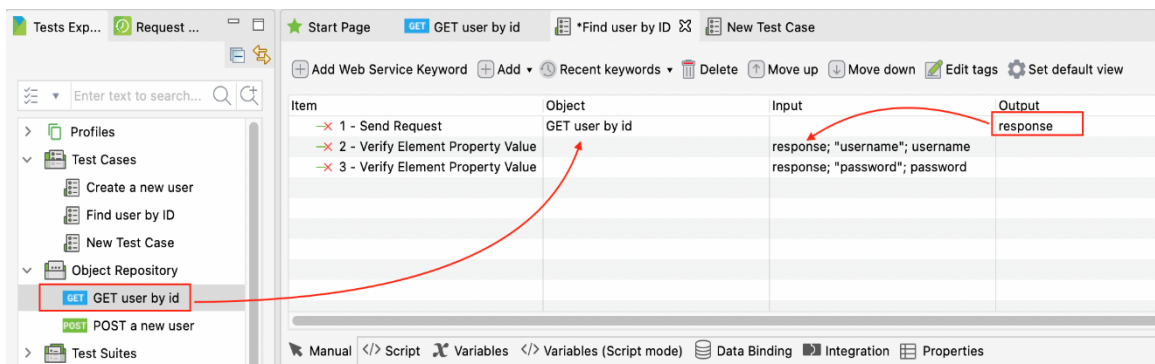
While the request at **Object Repository** is helpful for fast testing, we can add the request verification at the test case level for better managing and reporting.

3. Add an existing request to a test case.

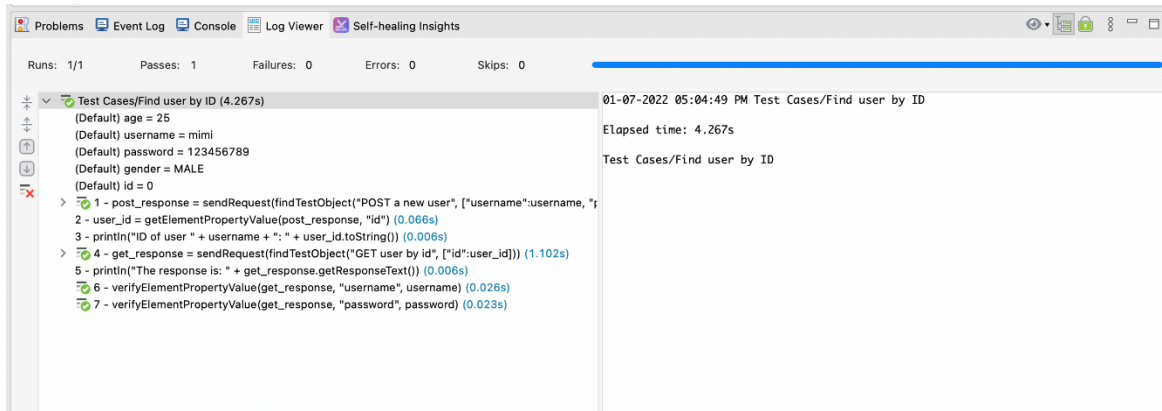
A request can be inserted into a test case with Web service built-in keywords. There are many keywords can be used to send the request, to verify the response, and to make the request as part of a bigger testing flow.



Following test case illustrates how we can call the request with verification steps from a test case:

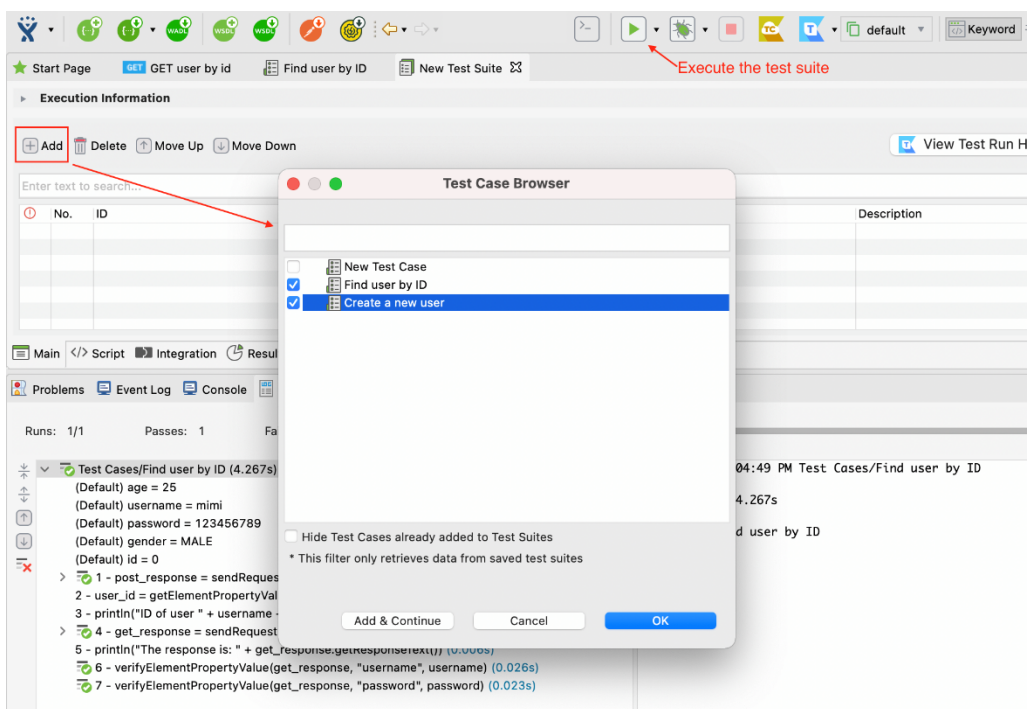


The test case can be executed as a normal test case in Katalon Studio. Each verification step can be viewed from the log.



4. Add the test case to the test suite.

A test case can be added to a test suite via either the drag-and-drop feature or the **Add test case** tool. Once the test case is added to the test suite, we can execute the whole test suite with the Run button (without selecting the browser as in UI testing).



Exercise:

1. Use <https://reqres.in/api> and create Testcase to execute Get, Put, Patch, Post and delete requests also use variable to define url. /*attach all printout here*/

2. state all status codes along with query types.

